

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 30 maja 2026

Spis treści

Streszczenie	6
1 Wprowadzenie	7
1.1 Motywacja i cel pracy	7
1.2 Zakres pracy	7
1.3 Struktura pracy	8
2 Analiza wymagań i przegląd rozwiązań	9
2.1 Wymagania funkcjonalne	9
2.2 Wymagania нефункционалне	10
2.3 Przegląd istniejących rozwiązań	11
2.3.1 OctoPrint	11
2.3.2 Mainsail oraz Fluidđ	11
2.3.3 Platformy komercyjne	12
2.3.4 Wnioski końcowe	12
3 Wybór technologii	13
3.1 Język programowania oraz środowisko uruchomieniowe	13
3.1.1 TypeScript oraz Node.js	13
3.1.2 Python	13
3.2 Framework webowy — Express.js	13
3.3 Baza danych — Azure Cosmos DB	14
3.4 Repozytorium obiektowe — Azure Blob Storage	14
3.5 Broker komunikatów — Azure Service Bus	14
3.6 Sterowanie urządzeniem — Azure IoT Hub	15
3.7 Interfejs użytkownika — React oraz Vite	15
3.8 Konteneryzacja — Docker oraz Docker Compose	16
3.9 Automatyzacja infrastruktury chmurowej — Terraform	16
3.10 Środowisko hostingowe backendu — Oracle Cloud Infrastructure	17
3.11 CAD - SolidWorks	17
3.12 Porównanie wybranych alternatyw	17

4	Architektura systemu	20
4.1	Ogólna struktura	20
4.2	Diagram architektury	20
4.3	Model danych	22
4.3.1	Użytkownik (kontener users)	23
4.3.2	Zadanie do wydruku (kontener jobs)	24
4.3.3	Token odświeżania (kontener refresh-tokens)	24
4.4	Cykl życia zadania do wydruku	25
4.5	Bezpieczeństwo i uwierzytelnianie	26
5	Implementacja serwisów backendowych	28
5.1	Auth Service	28
5.1.1	Opis ogólny	28
5.1.2	Punkty końcowe API	28
5.1.3	Kluczowe fragmenty implementacji	29
5.1.4	Weryfikacja adresu poczty elektronicznej	31
5.2	User Service	31
5.2.1	Opis ogólny	31
5.2.2	Punkty końcowe API	32
5.2.3	Mechanizm autoryzacji żądań	33
5.3	Files Service	35
5.3.1	Opis ogólny	35
5.3.2	Procedura przetwarzania i przesyłania plików	35
5.4	Queue Service	37
5.4.1	Opis ogólny	37
5.4.2	Mechanizm nasłuchiwanie kolejki	37
5.4.3	Punkty końcowe HTTP API	39
5.5	Printer Service	39
5.5.1	Opis ogólny	39
5.5.2	Mechanizm harmonogramowania	40
5.5.3	Monitor gotowości urządzenia	42
5.5.4	Odbiornik telemetry	42
5.5.5	Punkty końcowe HTTP API	43

5.6	Video Service	45
5.6.1	Opis ogólny	45
5.6.2	Punkty końcowe API	45
6	Implementacja warstwy prezentacji	46
6.1	Struktura architektoniczna aplikacji klienckiej	46
6.2	Scentralizowany klient API	47
6.3	Mechanizm routingu i ochrona tras	48
6.4	Ekrany aplikacji klienckiej	49
6.4.1	Strona główna systemu	49
6.4.2	Panel użytkownika	50
6.4.3	Przesyłanie i planowanie zadań	50
6.4.4	Kontrola procesu drukowania	51
6.4.5	Panel administracyjny	52
6.5	Internacjonalizacja aplikacji	53
6.6	Zarządzanie motywami graficznymi interfejsu	53
7	Agent sprzętowy — AddiPi Agent	55
7.1	Opis ogólny	55
7.2	Struktura obiektowa i architektura klas	55
7.3	Połączenie z IoT Hub	56
7.4	Obsługiwane metody bezpośrednie	56
7.5	Przebieg obsługi metody bezpośredniej startPrint	57
7.6	Transmisja danych telemetrycznych	58
7.7	Niskopoziomowa komunikacja z serwerem OctoPrint	58
8	Projekt mechaniczny obudowy i pozycjonowania zestawu	60
8.1	Cel i zakres projektu mechanicznego	60
8.2	Moduł CASE — obudowa Raspberry Pi z kamerą	60
8.2.1	Podstawa projektowa	60
8.2.2	Zakres modyfikacji geometrycznych	61
8.2.3	Integracja strukturalna i mocowanie modułu kamery brzegowej	61
8.2.4	Struktura plików i organizacja zasobów modułu CASE	62
8.3	Moduł STAND — regulowany mechanizm pozycjonujący	62

8.4	Złożenie kompletnego modelu geometrycznego	63
8.5	Parametry wydruku	64
8.6	Procedura montażu zestawu mechanicznego	65
9	Konfiguracja Raspberry Pi	66
9.1	Opis środowiska	66
9.2	Instalacja agenta	66
9.3	Automatyzacja uruchamiania za pomocą systemd	67
9.4	Udostępnianie strumienia wideo przez Cloudflare Tunnel	68
9.4.1	Instalacja narzędzia cloudflared	68
9.4.2	Skrypt automatyzacji publikacji adresu sieciowego	69
9.4.3	Automatyzacja odnawiania adresu w harmonogramie cron	70
9.5	Konfiguracja harmonogramu zadań crontab	70
9.5.1	Uzasadnienie zastosowanych parametrów	71
9.6	Problem dostępności sieci podczas uruchamiania systemu	71
9.7	Podsumowanie konfiguracji węzła brzegowego	72
10	Wdrożenie i infrastruktura produkcyjna	74
10.1	Hybrydowa strategia architektoniczna	74
10.2	Zarządzane usługi integracji danych i komunikacji chmurowej	74
10.3	Automatyzacja orkiestracji chmurowej — Terraform	75
10.4	Specyfikacja konteneryzacji i optymalizacja obrazów	76
10.5	Automatyzacja procesów CI/CD i Continuous Deployment	77
10.6	Orkiestracja kontenerowa warstwy serwerowej — Docker Compose	78
10.7	Zarządzanie konfiguracją i sekretami przez zmienne środowiskowe	80
10.8	Wdrożenie warstwy prezentacji oraz usługi wideo	80
10.9	Wdrożenie komponentu brzegowego	81
11	Podsumowanie i wnioski końcowe	82
11.1	Osiągnięte rezultaty i wkład inżynierski	82
11.2	Analiza napotkanych trudności projektowo-implementacyjnych	84
11.3	Ograniczenia przyjętego rozwiązania	86
11.4	Perspektywy i potencjalne kierunki dalszego rozwoju systemu	87
11.5	Wnioski końcowe	88

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz walidację rozproszonego systemu zarządzania procesami druku trójwymiarowego o nazwie *AddiPi*, dedykowanego dla akademickich laboratoriów studenckich. Opracowane rozwiązanie umożliwia użytkownikom zdalne przysyłanie plików z kodem sterującym (G-code), harmonogramowanie oraz monitorowanie stanu realizacji zadań w czasie rzeczywistym. Administratorom systemu udostępniono natomiast moduły pełnej kontroli nad kolejką produkcyjną, urządzeniami oraz profilami użytkowników.

Architektura systemu została oparta na paradygmacie mikroserwisowym. W strukturze backendowej wyodrębniono sześć niezależnych komponentów: usługę uwierzytelniania (*Auth Service*), zarządzania profilami (*User Service*), obsługi plików (*Files Service*), orkiestracji kolejki zadań (*Queue Service*), sterowania urządzeniem (*Printer Service*) oraz agenta sprzętowego (*AddiPi Agent*) uruchamianego na platformie Raspberry Pi. Warstwę prezentacji stanowi aplikacja kliencka (frontendowa) zbudowana z wykorzystaniem biblioteki React, którą uzupełnia dedykowany serwis wideo (*Video Service*) odpowiedzialny za dystrybucję strumienia wizyjnego.

Komunikacja wewnątrz systemu realizowana jest hybrydowo — za pomocą synchronicznego protokołu HTTP (architektura REST API) oraz asynchronicznej, chmurowej kolejki *Azure Service Bus*. Dane aplikacyjne są utrwalane w wielomodelowej bazie danych *Azure Cosmos DB*, a pliki G-code w repozytorium obiektowym *Azure Blob Storage*. Zarządzanie i sterowanie fizyczną drukarką 3D odbywa się za pośrednictwem usługi *Azure IoT Hub* przy użyciu mechanizmu metod bezpośrednich (ang. *Direct Methods*) wywoływanych na mikrokomputerze, na którym osadzono oprogramowanie serwerowe *OctoPrint*.

Poszczególne serwisy zaimplementowano w językach TypeScript (środowisko Node.js) oraz Python. Proces wdrożenia środowiska oparto na technologii konteneryzacji Docker, koordynowanej za pomocą narzędzia Docker Compose.

Słowa kluczowe: mikroserwisy, drukowanie 3D, OctoPrint, Raspberry Pi, Azure IoT Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, systemy rozproszone.

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym — ich liczba w danej jednostce jest zwykle ograniczona, a czas wytwarzania pojedynczego modelu fizycznego może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne wyzwania logistyczne i organizacyjne, takie jak konieczność ręcznego rezerwowania terminów, brak przejrzystości w strukturze kolejki zadań oraz brak możliwości zdalnego monitorowania stanu operacyjnego maszyn.

Głównym celem niniejszej pracy jest zaprojektowanie, implementacja oraz walidacja rozproszonego systemu *AddiPi*, którego zadaniem jest pełna automatyzacja i orkiestracja procesów zarządzania drukarkami 3D w środowisku laboratorium studenckiego. Opracowane oprogramowanie udostępnia użytkownikom funkcjonalności w zakresie:

- asynchronicznego przesyłania plików z kodem sterującym (G-code) za pośrednictwem aplikacji klienckiej,
- elastycznego harmonogramowania procesów druku na określony termin lub lokowania ich w ogólnej kolejce systemu,
- śledzenia bieżących metryk postępu prac w czasie rzeczywistym wraz z bezpośrednim podglądem wizyjnym,
- automatycznego otrzymywania powiadomień o pomyślnym zakończeniu bądź awarii procesu produkcyjnego.

Personelowi administracyjnemu laboratorium system zapewnia dedykowane narzędzia do zarządzania profilami użytkowników i ich uprawnieniami, audytu historii wszystkich operacji oraz zdalnego sterowania infrastrukturą sprzętową i priorytetami kolejki.

1.2 Zakres pracy

Zakres rzeczowy zrealizowanego projektu inżynierskiego obejmuje:

1. przeprowadzenie analizy wymagań funkcjonalnych i нефункциональных oraz wybór technologii,
2. opracowanie projektu rozproszonej architektury mikroserwisowej,
3. implementację komponentów serwerowych (backendowych) oraz aplikacji klienckiej (frontendowej),
4. konfigurację i integrację dedykowanej infrastruktury chmurowej w modelu PaaS (*Microsoft Azure*),
5. konteneryzację oraz automatyzację procesu wdrożenia systemu przy użyciu narzędzi *Docker* oraz *Docker Compose*,
6. przeprowadzenie testów weryfikacyjnych i walidacji poprawności działania całego ekosystemu.

1.3 Struktura pracy

Niniejsza praca dyplomowa została podzielona na logiczne segmenty, których zawartość odzwierciedla kolejne etapy realizacji projektu. Rozdział 2 zawiera szczegółową analizę wymagań funkcjonalnych i нефункциональных oraz krytyczny przegląd istniejących rozwiązań rynkowych. W rozdziale 3 omówiono wybrane technologie programistyczne i chmurowe wraz z technicznym uzasadnieniem podjętych decyzji projektowych. Projekt samej architektury rozproszonej oraz przepływy danych w systemie zostały opisane w rozdziale 4.

Szczegółowa implementacja poszczególnych komponentów programowych, z uwzględnieniem warstwy serwerowej, klienckiej oraz komponentów pomocniczych, stanowi treść rozdziałów od 5 do 7. Z kolei w rozdziale 8 przedstawiono proces projektowania mechanicznego i trójwymiarowego modelowania obudowy urządzenia w programie *SolidWorks*. Specyfika konfiguracji mikrokomputera sterującego, automatyzacja zadań systemowych przez mechanizm *cron* oraz konfiguracja bezpiecznego tunelu sieciowego *Cloudflare* zostały przybliżone w rozdziale 9.

Końcowe etapy prac, obejmujące orkiestrację infrastruktury chmurowej *Microsoft Azure* oraz procesy konteneryzacji środowiska, opisano w rozdziale 10. Całość pracy kończy się podsumowaniem oraz wnioskami końcowymi, zawartymi w rozdziale 11.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie szczegółowej analizy specyfiki oraz potrzeb akademickiego laboratorium studenckiego sformułowano następujące wymagania funkcjonalne, z podziałem na rolę użytkownika standardowego oraz administratora systemu.

W obszarze kont użytkowników (rola standardowa):

- rejestracja konta powiązana z dwuetapową weryfikacją adresu e-mail, ograniczoną wyłącznie do domeny akademickiej @uwr.edu.pl,
- bezpieczne uwierzytelnianie i autoryzacja z wykorzystaniem mechanizmu podwójnych tokenów JWT (*access token* oraz *refresh token*) [1],
- asynchroniczne przesyłanie plików z kodem sterującym (G-code) o maksymalnym rozmiarze pojedynczego wolumenu do 50 MB i rozszerzeniu .gcode,
- elastyczne harmonogramowanie procesów wytwórczych na wybrany termin kalendarzowy lub lokowanie ich w ogólnej kolejce systemu,
- przeglądanie historii własnych zleceń produkcyjnych z opcją dynamicznego filtrowania według stanów maszyny,
- monitorowanie parametrów środowiskowych i postępu procesu druku 3D w czasie rzeczywistym (weryfikacja paska postępu, temperatury dyszy oraz stołu roboczego, a także podgląd wizyjny z kamery),
- możliwość manualnego anulowania lub ponownego inicjowania własnych zadań.

W obszarze administracji i nadzoru (rola administratora):

- wgląd w rejestr kont oraz zarządzanie profilami wszystkich użytkowników systemu,
- dynamiczna modyfikacja poziomu uprawnień i ról użytkowników (nadawanie statusu user lub admin),
- kompleksowe zarządzanie globalną kolejką oraz historią wszystkich zarejestrowanych zadań drukowania,

- zdalne potwierdzanie fizycznej i operacyjnej gotowości urządzenia do podjęcia kolejnego cyklu produkcyjnego,
- dostęp do zagregowanych metryk systemowych, obejmujących m.in. statystyki zadań z podziałem na statusy.

2.2 Wymagania niefunkcjonalne

W celu zapewnienia odpowiedniej jakości, wydajności oraz stabilności działania rozproszonej platformy informatycznej, określono następujące wymagania jakościowe (niefunkcjonalne):

- **Skalowalność** — architektura rozproszona powinna umożliwiać bezproblemową integrację i rejestrację kolejnych fizycznych urządzeń drukujących bez konieczności modyfikacji lub przebudowy rdzenia systemu.
- **Niezawodność i tolerancja błędów** — asynchroniczne przetwarzanie komunikatów z wykorzystaniem magistrali powinno gwarantować, że chwilowa niedostępność komponentów nie spowoduje trwałej utraty zlecenia druku ani niespójności stanów w bazie danych.
- **Bezpieczeństwo danych** — autoryzacja dostępu do interfejsów programistycznych REST API realizowana za pomocą kryptograficznie podpisanych tokenów JWT, natomiast hasła użytkowników bezwzględnie utrwalane w bazie danych w postaci kryptograficznych skrótów (ang. *hashes*) wygenerowanych algorytmem *bcrypt*.
- **Responsywność interfejsu** — warstwa prezentacji (aplikacja frontendowa) musi zostać zaprojektowana zgodnie z paradygmatem RWD (ang. *Responsive Web Design*), zapewniając pełną ergonomię i poprawność wyświetlania zarówno na urządzeniach stacjonarnych, jak i mobilnych.
- **Wydajność czasowa** — czas odpowiedzi punktów końcowych API na zapytania synchroniczne nie powinien przekraczać 2 sekund w warunkach standardowego, nominalnego obciążenia infrastruktury.
- **Przenośność i konteneryzacja** — wszystkie komponenty, mikroserwisy oraz bazy danych wchodzące w skład systemu muszą zostać spakowane w niezależne konte-

nery środowiska *Docker*, umożliwiając pełne uruchomienie całego ekosystemu za pomocą jednego polecenia skryptowego (*Docker Compose*).

2.3 Przegląd istniejących rozwiązań

2.3.1 OctoPrint

Oprogramowanie *OctoPrint* [2] stanowi obecnie jedno z najpopularniejszych rozwiązań typu *open-source* dedykowanych do zdalnego zarządzania i monitorowania pojedynczych drukarek 3D. System ten, uruchamiany najczęściej jako usługa serwerowa w środowisku Linux na mikrokomputerze Raspberry Pi, udostępnia responsywny interfejs webowy, programistyczny interfejs REST API oraz rozbudowany ekosystem wtyczek (ang. *plugins*). Umożliwia on bezpośrednie kontrolowanie parametrów mechanicznych i termicznych drukarki, wizualizację kodu sterującego, podgląd wideo z kamery oraz analizę postępu prac.

Głównym ograniczeniem platformy *OctoPrint* jest jednak jej monolityczna architektura, zaprojektowana z myślą o pojedynczym użytkowniku zarządzającym wyłącznie jedną maszyną. Oprogramowanie to nie oferuje natywnych mechanizmów globalnego kolejkowania zadań dla wielu niezależnych kont, zaawansowanego systemu kontroli dostępu opartego na rolach (RBAC) ani integracji z infrastrukturą chmurową. Z tego względu w projekcie *AddiPi* system *OctoPrint* został wykorzystany wyłącznie w charakterze lokalnego, niskopoziomowego sterownika sprzętowego, nad którym nadbudowano rozproszoną warstwę orkiestracji i zarządzania.

2.3.2 Mainsail oraz Fluidd

Ekosystemy *Mainsail* oraz *Fluidd* to nowoczesne, wysoce zoptymalizowane interfejsy webowe współpracujące bezpośrednio z oprogramowaniem układowym (firmware) *Klipper*. Charakteryzują się one niskim narzutem wydajnościowym oraz zaawansowanymi algorytmami kompensacji drgań maszyn. Podobnie jednak jak w przypadku oprogramowania *OctoPrint*, ich architektura została zorientowana na scenariusz jednorodny (jeden operator — jedno urządzenie) i nie przewiduje mechanizmów współdzielenia zasobów sprzętowych ani wielodostępowej orkiestracji kolejki zadań.

2.3.3 Platformy komercyjne

Rozwiązania takie jak *3DprinterOS*, *Polar Cloud* czy *Ultimaker Digital Factory* to komercyjne systemy klasy *fleet management*, przeznaczone do scentralizowanego zarządzania rozproszoną flotą drukarek trójwymiarowych w sektorze przemysłowym oraz edukacyjnym. Platformy te udostępniają zaawansowane moduły analityczne i kolejki zadań, jednak ich implementacja wiąże się z wysokimi, cyklicznymi kosztami subskrypcyjnymi. Dodatkowo są to systemy zamknięte, działające w modelu SaaS (*Software as a Service*), co uniemożliwia pełną kontrolę nad integralnością i lokalizacją przetwarzanych danych — w specyficznych warunkach laboratoriów akademickich stanowi to istotną barierę wdrożeniową.

2.3.4 Wnioski końcowe

Przeprowadzona analiza wykazała, że żadne z dostępnych na rynku rozwiązań o otwartym kodzie źródłowym nie łączy w sobie wielodostępowego systemu uwierzytelniania, asynchronicznego kolejkowania zadań z funkcją planowania kalendarzowego, bezpośredniej integracji z chmurą obliczeniową oraz sterowania bazującego na paradygmacie Internetu Rzeczy (IoT). Projekt *AddiPi* skutecznie wypełnia tę lukę technologiczną. Stanowi on autonomiczne, bezpieczne i wysoce elastyczne środowisko, które pozwala na pełną niezależność infrastrukturalną przy jednoczesnym zachowaniu niskich kosztów utrzymania systemu.

3 Wybór technologii

3.1 Język programowania oraz środowisko uruchomieniowe

3.1.1 TypeScript oraz Node.js

Większość serwisów backendowych oraz aplikację frontendową zaimplementowano w języku TypeScript [3], będącym nadzbiorem języka JavaScript, który wprowadza statyczne typowanie. Wybór ten znacząco ułatwia utrzymanie rozbudowanych baz kodu poprzez wykrywanie błędów na etapie kompilacji, zaawansowane wsparcie środowisk programistycznych (IDE) oraz samodokumentowanie struktury danych za pomocą typów.

Platformę Node.js [4] wybrano ze względu na asynchroniczny, nieblokujący model wejścia-wyjścia. Rozwiązanie to charakteryzuje się wysoką wydajnością w systemach obsługujących liczne, równoległe połączenia HTTP oraz intensywną komunikację z usługami zewnętrznymi, takimi jak bazy danych, kolejki komunikatów czy usługa *Azure IoT Hub*.

3.1.2 Python

Agent działający na platformie Raspberry Pi został zaimplementowany w języku Python [5]. Biblioteki *azure-iot-device* oraz *azure-storage-blob* są w tym środowisku powszechnie dostępne i szczegółowo udokumentowane. Python stanowi standardowe rozwiązanie w przypadku aplikacji uruchamianych na systemach wbudowanych oraz mikrokomputerach jednoukładowych (ang. *Single-Board Computer*, SBC).

3.2 Framework webowy — Express.js

Wszystkie serwisy warstwy Node.js wykorzystują framework Express.js [6] w wersji stabilnej 5.0. Rozwiązanie to charakteryzuje się minimalistyczną architekturą i niskim narzutem wydajnościowym. Framework nie narzuca sztywnej struktury katalogów (ang. *unopinionated framework*), co pozwala na elastyczne dostosowanie architektury każdego z mikroservisów do jego specyficznych wymagań. Express.js zapewnia również ustandaryzowane mechanizmy obsługi żądań pośredniczących (ang. *middleware*), routingu oraz zarządzania błędami.

3.3 Baza danych — Azure Cosmos DB

Azure Cosmos DB [7] to w pełni zarządzana, wielomodelowa baza danych typu NoSQL, oferująca globalną dystrybucję danych, elastyczne partycjonowanie oraz natywną obsługę zróżnicowanych interfejsów programistycznych (SQL, MongoDB, Cassandra, Gremlin, Table). W projekcie *AddiPi* wykorzystano interfejs API SQL (Core), który umożliwia wykonywanie zapytań w standardzie zbliżonym do języka SQL na dokumentach w formacie JSON.

Wybór tej technologii został podyktowany następującymi czynnikami:

- **Model usługowy PaaS (Platform as a Service)** — pełne zarządzanie infrastrukturą przez dostawcę chmurowego eliminuje konieczność administracji serwerem bazy danych,
- **Elastyczny schemat danych** — poszczególne mikroserwisy przechowują obiekty o zróżnicowanej, heterogenicznej strukturze w ramach jednej bazy danych (*addipi*), z podziałem na dedykowane kontenery (*users*, *jobs*, *refresh-tokens*),
- **Natywna integracja** — bezszwowe współdziałanie z pozostałymi komponentami ekosystemu chmurowego *Microsoft Azure*,
- **Automatyczne indeksowanie** — domyślne indeksowanie wszystkich pól w strukturze dokumentu, co optymalizuje czas wykonywania zapytań.

3.4 Repozytorium obiektowe — Azure Blob Storage

Pliki G-code przesyłane przez użytkowników są przechowywane w usłudze repozytorium obiektowego *Azure Blob Storage* [8] w ramach dedykowanego kontenera o nazwie *gc ode*. Usługa ta zapewnia niemal nieograniczoną przestrzeń magazynową, wysoki poziom dostępności, relatywnie niskie koszty utrzymania danych oraz zoptymalizowany interfejs API dedykowany do bezpiecznego przesyłania i pobierania dużych obiektów binarnych (ang. *Binary Large Objects*, BLOB).

3.5 Broker komunikatów — Azure Service Bus

Azure Service Bus [9] to chmurowa usługa kolejkowania komunikatów i zarządzania strumieniami danych klasy korporacyjnej. W architekturze systemu *AddiPi* pełni ona klu-

czową rolę brokera komunikatów (ang. *message broker*) odpowiedzialnego za asynchroniczne i bezprzerwowe pośrednictwo pomiędzy usługą *Files Service* (producentem komunikatów) a komponentem *Queue Service* (konsumentem).

Dedykowana kolejka print-queue zapewnia niezawodną semantykę dostarczania wiadomości typu *at-least-once* (co najmniej raz). Rozwiązanie to gwarantuje, że w przypadku awarii usługi konsumenckiej lub utraty połączenia sieciowego, komunikat nie zostanie skasowany — pozostaje on zabezpieczony w strukturze magistrali chmurowej i może zostać przetworzony powtórnie po przywróceniu operacyjności konsumenta, co bezpośrednio realizuje wymaganie niefunkcjonalne dotyczące tolerancji błędów.

3.6 Sterowanie urządzeniem — Azure IoT Hub

Azure IoT Hub [10] to usługa chmurowa przeznaczona do scentralizowanego zarządzania, bezpiecznego uwierzytelniania oraz monitorowania urządzeń Internetu Rzeczy (IoT). Usługa ta umożliwia pełną dwukierunkową komunikację (ang. *Device-to-Cloud* oraz *Cloud-to-Device*), a także udostępnia mechanizm metod bezpośrednich (ang. *Direct Methods*). Metody te stanowią synchroniczny wzorzec wywołań typu RPC (ang. *Remote Procedure Call*), realizowany na urządzeniach brzegowych komunikujących się za pośrednictwem protokołów MQTT lub AMQP.

W architekturze systemu *AddiPi* usługa *Printer Service* zdalnie wywołuje na agencie (uruchomionym na platformie Raspberry Pi) metody bezpośrednio: `startPrint`, `cancelPrint` oraz `getStatus`. W odpowiedzi zwrotnej agent transmituje asynchroniczny strumień danych telemetrycznych w postaci zdarzeń systemowych (m.in. `print_started`, `print_progress` czy `print_completed`). Komunikaty te są agregowane przez usługę *Printer Service* za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*.

3.7 Interfejs użytkownika — React oraz Vite

Warstwa prezentacji (interfejs użytkownika) została zaimplementowana z wykorzystaniem biblioteki *React* [11] w wersji 18 oraz języka TypeScript. W charakterze środowiska deweloperskiego oraz narzędzia do automatyzacji procesu budowania (ang. *bundler*) aplikacji wykorzystano framework *Vite* [12], co pozwoliło na optymalizację czasu kompilacji kodu źródłowego.

Do zarządzania globalnym stanem aplikacji klienckiej zastosowano bibliotekę *Zustand* [13]. Rozwiązanie to umożliwia wydajne współdzielenie danych pomiędzy komponentami drzewa DOM w sposób bezstanowy, eliminując konieczność implementowania nadmiarowej i złożonej architektury (takiej jak *Redux*) przy jednoczesnym zachowaniu wysokiej reaktywności interfejsu.

Projekt graficzny oraz warstwa stylów zostały zrealizowane przy użyciu frameworka *Tailwind CSS* [14]. Narzędzie to bazuje na architekturze klas narzędziowych (ang. *utility-first*), w której style interfejsu są komponowane bezpośrednio w strukturze kodu za pomocą predefiniowanych klas reprezentujących pojedyncze właściwości kaskadowych arkuszy stylów.

3.8 Konteneryzacja — Docker oraz Docker Compose

W celu zapewnienia powtarzalności środowiska uruchomieniowego, każdy z mikroservisów wchodzących w skład ekosystemu posiada dedykowany plik konfiguracyjny *Dockerfile*. Proces budowania obrazów realizuje mechanizm wieloetapowy (ang. *multi-stage build*). Podejście to pozwala na drastyczne ograniczenie rozmiaru finalnych obrazów kontenerów oraz całkowite odizolowanie środowiska kompilacji i zależności deweloperskich od produkcyjnego środowiska uruchomieniowego.

Narzędzie *Docker Compose* [15] wykorzystano do zautomatyzowanej orkiestracji i zarządzania cyklem życia wszystkich kontenerów zarówno w środowisku lokalnym, jak i produkcyjnym. Narzędzie to definiuje odizolowaną, wspólną sieć wirtualną *addip-network* oraz odpowiada za bezpieczne wstrzykiwanie zmiennych środowiskowych odczytywanych z pliku *.env*.

3.9 Automatyzacja infrastruktury chmurowej — Terraform

Do zarządzania rozproszonymi zasobami chmurowymi wykorzystano narzędzie *Terraform* [16], implementujące paradygmat deklaratywnego definiowania środowisk systemowych jako kod (ang. *Infrastructure as Code*, IaC). Architektura wdrożeniowa została podzielona na dwa niezależne moduły: *terraform/* dedykowany dla usług platformy *Microsoft Azure* (z użyciem providera *hashicorp/azurerm*) oraz *terraform-oci/* konfigurujący zasoby w chmurze *Oracle Cloud Infrastructure* (oparty na providerze *oracle-oci*). Wybór tego rozwiązania gwarantuje pełną powtarzalność środowisk produkcyj-

nych oraz automatyzację procesów aprowizacji.

3.10 Środowisko hostingowe backendu — Oracle Cloud Infrastructure

Kontenery aplikacyjne warstwy serwerowej (*Auth*, *User*, *Files*, *Queue* oraz *Printer Service*) są uruchamiane na maszynie wirtualnej wewnątrz chmury *Oracle Cloud Infrastructure* (OCI) [17]. W projekcie wykorzystano instancję `VM.Standard.E2.1.Micro` udostępnianą w bezpłatnym programie *Always Free* (specyfikacja sprzętowa: 1 OCPU, 1 GB pamięci RAM). Platformę OCI wybrano z uwagi na dostępność autonomicznych zasobów obliczeniowych działających bez restrykcji czasowych, co stanowi ekonomicznie optymalną alternatywę dla rozwiązań komercyjnych rozliczanych w modelu subskrypcyjnym lub za faktyczny czas pracy instancji.

3.11 CAD - SolidWorks

Projekt mechaniczny obudowy urządzenia wykonano w programie **SolidWorks** [18], stanowiącym środowisko do parametrycznego modelowania bryłowego oraz projektowania złożeń mechanicznych. Oprogramowanie umożliwia tworzenie asocjatywnych modeli 3D, w których modyfikacja parametrów części powoduje automatyczną aktualizację złożeń oraz dokumentacji technicznej.

Wybór programu SolidWorks był podyktowany dostępnością oprogramowania w ramach licencji studenckiej oraz szerokim wsparciem dla formatów eksportu modeli, takich jak `.STEP` (neutralny format wymiany modeli CAD), `.STL` (format wykorzystywany w procesie przygotowania modeli o druku 3D) oraz `.3MF` (format wymiany zawierający dodatkowe metadane procesu druku).

Projekt został podzielony na pliki `.SLDPRT`, reprezentujące pojedyncze części, oraz `.SLDASM`, definiujące kompletne złożenia mechaniczne.

3.12 Porównanie wybranych alternatyw

Proces projektowania systemu wymagał wielokryterialnej analizy dostępnych rozwiązań technologicznych. Decyzje architektoniczne oparto na kryteriach takich jak: stopień integracji z chmurą obliczeniową, łatwość skalowania, nakłady na zarządzanie infrastrukturą oraz spójność ekosystemu programistycznego. Zbiornicze zestawienie wszystkich podję-

tych decyzji projektowych, wybranych technologii oraz odrzuconych alternatyw przedstawiono na rysunku 1.

Kategoria	Wybrana technologia	Odrzucone alternatywy	Kluczowy powód
Język backendowy serwisy Node.js + agent	TypeScript / Node.js + Python (agent RPi) Express.js v5	Go, Java, PHP	statyczne typowanie, duży ekosystem npm, nieblokujące I/O
Baza danych users, jobs, tokens	Azure Cosmos DB SQL API (JSON) PaaS, bez serwera	PostgreSQL, MongoDB Atlas	natywna integracja z Azure, elastyczny schemat JSON
Kolejka komunikatów upload → zadanie	Azure Service Bus kolejka print-queue at-least-once, PaaS	RabbitMQ, Apache Kafka	brak zarządzania serwerem, auto- skalowanie
Sterowanie IoT chmura → Raspberry Pi	Azure IoT Hub Direct Methods, MQTT telemetria D2C	AWS IoT Core, MQTT broker własny	Direct Methods, ekosystem Azure, telemetria wbudowana
Frontend aplikacja webowa	React 18 + Vite TypeScript, Zustand Tailwind CSS, i18next	Vue, Angular, Next.js	dojrzały ekosystem, hooks, SPA bez SSR (prostota)
Wdrożenie konteneryzacja	Docker + Compose multi-stage build addip-network	Kubernetes, Podman, bare metal	przenośność, jedna komenda uruchomienia, izolacja serwisów

Rysunek 1: Zbiorne zestawienie wybranej architektury technologicznej, odrzuconych alternatyw oraz kluczowych powodów decyzyjnych.

Głównym założeniem przy wyborze warstwy backendowej oraz frontendowej było ujednolicenie stosu technologicznego wokół języka TypeScript oraz środowiska Node.js. Pozwoliło to na współdzielenie typów danych między aplikacją webową (React 18 + Vite) a usługami serwerowymi, co znacząco przyspieszyło proces implementacji. Wyjątek stanowi mikrousluga *Files Service* oraz komponent agenta uruchamiany na platformie Raspberry Pi, gdzie ze względu na specyfikę obsługi interfejsów sprzętowych zdecydowano się na wykorzystanie języka Python.

Kluczowe znaczenie dla stabilności systemu miało precyzyjne dobranie silnika bazy danych oraz mechanizmu wymiany komunikatów. Szczegółowe porównanie cech rozważanych systemów bazodanowych oraz systemów kolejkowych zestawiono odpowiednio w tabeli 1 oraz tabeli 2.

Tabela 1: Porównanie baz danych rozważanych w projekcie

Cecha	Cosmos DB	PostgreSQL	MongoDB Atlas
Model danych	Dokument (JSON)	Relacyjny	Dokument (BSON)
Zarządzanie	PaaS (bez serwera)	Samodzielny/PaaS	PaaS
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Elastyczny schemat danych	Tak	Nie	Tak
Dostępność bezpłatnego planu	Tak (400 RU/s)	Tak (self-hosted)	Tak (512 MB)

Jak wynika z tabeli 1, baza Azure Cosmos DB została wybrana ze względu na bezobsługowy model bezserwerowy (Serverless PaaS) oraz natywną integrację z ekosystemem Azure. Elastyczny schemat JSON idealnie odpowiada dynamicznej strukturze danych generowanych przez sensory IoT, eliminując narzut związany z migracjami schematu w tradycyjnych bazach relacyjnych (takich jak PostgreSQL).

Tabela 2: Porównanie rozwiązań kolejkowania wiadomości

Cecha	Azure Service Bus	RabbitMQ	Apache Kafka
Model wdrożenia	PaaS	Self-hosted	Self-hosted / PaaS
Semantyka dostarczania	At-least-once	At-least-once	At-least-once
Złożoność konfiguracji	Niska	Średnia	Wysoka
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Skalowalność	Automatyczna	Ręczna	Wysoka

W przypadku infrastruktury kolejki (tab. 2), usługa Azure Service Bus przewyższyła alternatywy brakiem konieczności samodzielnego zarządzania serwerem oraz wbudowaną automatyczną skalowalnością. Narzut konfiguracyjny rozwiązań takich jak Apache Kafka okazał się nieuzasadniony w stosunku do skali projektu, natomiast Azure Service Bus zapewnił wymaganą semantykę dostarczania wiadomości *at-least-once* bezpośrednio po uruchomieniu usługi.

Ostatnim etapem konfiguracji było zdefiniowanie środowiska uruchomieniowego. Odrzucono złożone systemy orkiestracji (Kubernetes) oraz instalacje bezpośrednie (bare metal) na rzecz narzędzia Docker + Compose. Zapewniło to pełną przenośność kodu, powtarzalność wdrożeń przy użyciu wieloetapowego budowania obrazów (*multi-stage build*) oraz bezpieczną izolację usług w ramach dedykowanej sieci wirtualnej o nazwie `addip-network`.

4 Architektura systemu

4.1 Ogólna struktura

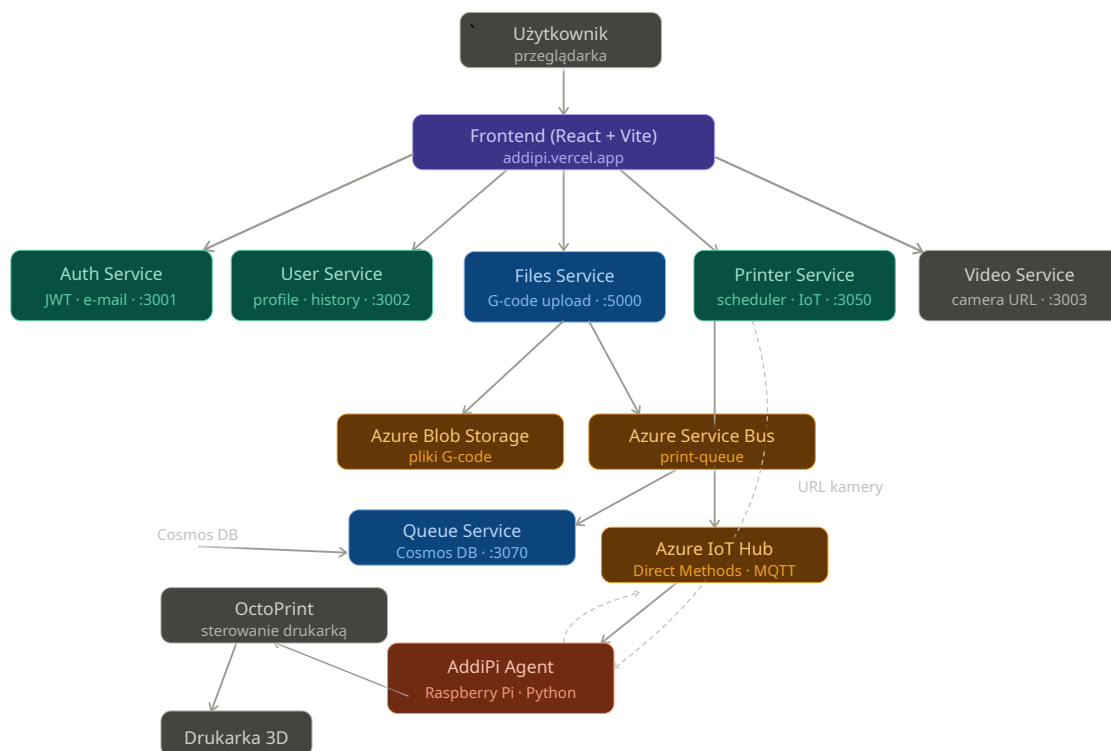
System AddiPi zbudowany jest w architekturze mikroserwisowej. Każdy serwis stanowi niezależną jednostkę wdrożeniową odpowiedzialną za określoną domenę biznesową oraz komunikuje się z pozostałymi komponentami za pośrednictwem zdefiniowanych interfejsów (HTTP REST lub kolejka komunikatów).

W ramach systemu wyodrębniono następujące komponenty:

1. **AddiPi Auth Service** (port 3001) - realizuje proces uwierzytelniania użytkowników oraz obsługę tokenów JWT.
2. **AddiPi User Service** (port 3002) - odpowiada za zarządzanie profilami użytkowników oraz historią zadań do wydruku.
3. **AddiPi Files Service** (port 5000) - umożliwia przesyłanie oraz przechowywanie plików G-code.
4. **AddiPi Queue Service** (porty 3070) - odbiera komunikaty z kolejki oraz zapisuje informacje o zadaniach w bazie Azure Cosmos DB.
5. **AddiPi Printer Service** (port 3050) - odpowiada za harmonogramowanie zadań do wydruku, komunikację z Azure IoT Hub oraz odbiór telemetrii urządzenia.
6. **AddiPi Video Service** (port 3003) - odpowiada za rejestrację i udostępnianie adresu URL kamery systemu OctoPrint.
7. **AddiPi Agent** (Raspberry Pi) - agent uruchamiany na urządzeniu Raspberry Pi, pośredniczący pomiędzy Azure IoT Hub a systemem OctoPrint.
8. **AddiPi Frontend** (Vercel / port 5173) - aplikacja webowa zbudowana z wykorzystaniem biblioteki React.

4.2 Diagram architektury

Na rysunku 2 przedstawiono sekwencyjny przepływ danych pomiędzy poszczególnymi komponentami zaprojektowanego systemu.



Rysunek 2: Diagram architektury systemu.

1. **Uwierzytelnianie użytkownika:** Użytkownik loguje się w systemie za pośrednictwem aplikacji klienckiej (frontendowej). Aplikacja nawiązuje komunikację z usługą *Auth Service*, która po pomyślnej weryfikacji tożsamości zwraca parę tokenów JWT (*JSON Web Token*).
2. **Przesyłanie pliku produkcyjnego:** Użytkownik przesyła plik z kodem sterującym (G-code). Aplikacja frontendowa wywołuje punkt końcowy (ang. *endpoint*) POST `/files/upload` w ramach usługi *Files Service*. Następnie komponent ten zapisuje plik w repozytorium *Azure Blob Storage* oraz publikuje komunikat `file_uploaded` w kolejce usługi *Azure Service Bus*.
3. **Inicjalizacja zadania w bazie danych:** Usługa *Queue Service* asynchronicznie odbiera komunikaty z kolejki i na ich podstawie tworzy dokument zadania w bazie danych *Azure Cosmos DB* (w ramach kontenera `jobs`), przypisując mu status początkowy `pending` lub `scheduled`.
4. **Harmonogramowanie i weryfikacja stanu urządzenia:** Usługa *Printer Service* w sposób cykliczny (z interwałem jednominutowym) weryfikuje dostępność zadań

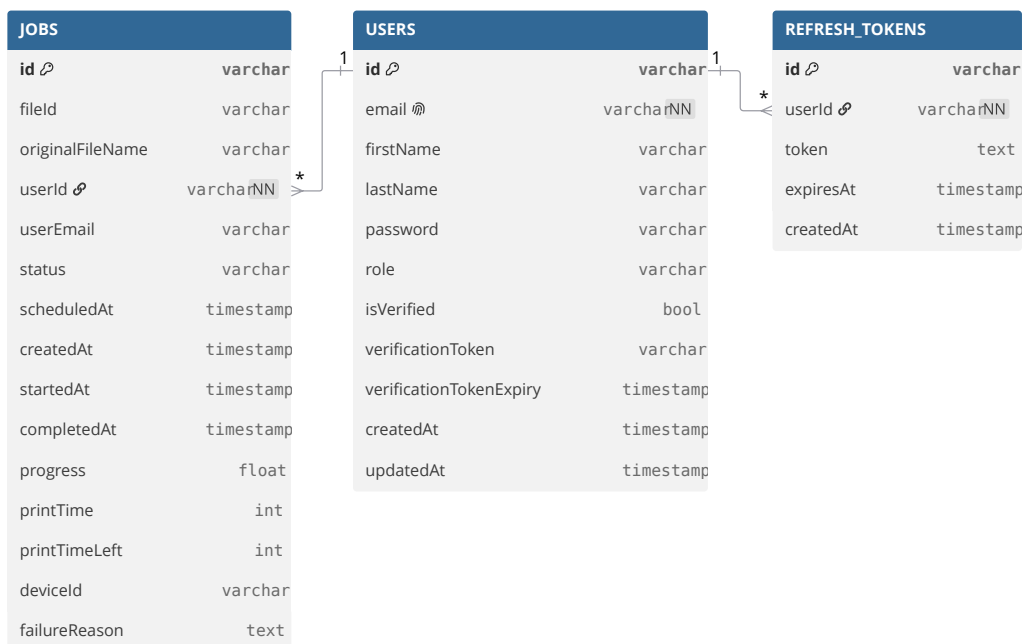
oczekujących na realizację. Jeżeli urządzenie drukujące jest wolne, a zadanie spełnia kryteria rozpoczęcia procesu wytwórczego, status transakcji jest zmieniany na `waiting_for_printer_ready`. System przechodzi wówczas w stan oczekiwania na potwierdzenie gotowości operacyjnej drukarki.

5. **Inicjacja komunikacji IoT:** Po autoryzacji gotowości urządzenia przez użytkownika lub administratora, usługa *Printer Service* zdalnie wywołuje metodę bezpośrednią (ang. *Direct Method*) `startPrint` na dedykowanym agencie, wykorzystując do tego celu usługę *Azure IoT Hub*.
6. **Zarządzanie procesem drukowania przez agenta:** Komponent *AddiPi Agent* pobiera przypisany plik G-code z repozytorium *Azure Blob Storage*, przekazuje go bezpośrednio do interfejsu oprogramowania *OctoPrint* oraz inicjuje proces drukowania 3D. W trakcie pracy agent w sposób ciągły transmituje dane telemetryczne do usługi *Azure IoT Hub*, raportując bieżący postęp, status ukończenia lub ewentualne błędy krytyczne.
7. **Odbiór telemetrii i aktualizacja stanu:** Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, a następnie aktualizuje stan procesu w bazie *Azure Cosmos DB*.
8. **Odświeżanie danych w interfejsie:** Aplikacja kliencka (frontendowa) realizuje cykliczne zapytania sondujące (ang. *polling* z interwałem 5 sekund) do mikroserwisów, zapewniając bieżącą aktualizację stanu systemu prezentowanego użytkownikowi.

4.3 Model danych

W bazie danych *Azure Cosmos DB* dane są strukturyzowane i przechowywane w postaci dokumentów w formacie JSON. Zostały one pogrupowane w logicznych kontenerach, odpowiadających poszczególnym domenom biznesowym systemu.

Na rysunku 3 przedstawiono fizyczny diagram encji i relacji (ang. *Entity-Relationship Diagram*, ERD) zaprojektowanej bazy danych *addipi-cosmos*. Diagram ten odwzorowuje logiczne powiązania referencyjne pomiędzy dokumentami w poszczególnych kontenerach.



Rysunek 3: Diagram bazy danych addipi-cosmos w notacji Crow's Foot (ptasiej łapki).

4.3.1 Użytkownik (kontener users)

Dokument przechowuje informacje profilowe użytkowników, dane autoryzacyjne oraz status weryfikacji konta. Przykładową strukturę przedstawiono na listingu 1.

```

1  {
2    "id": "user_<timestamp>",
3    "email": "student@uwr.edu.pl",
4    "firstName": "Jan",
5    "lastName": "Kowalski",
6    "password": "<bcrypt hash>",
7    "role": "user",
8    "isVerified": true,
9    "verificationToken": "<hex>",
10   "verificationTokenExpiry": "2025-01-01T00:00:00Z",
11   "createdAt": "2025-01-01T00:00:00Z",
12   "updatedAt": "2025-01-01T00:00:00Z"
13 }

```

Listing 1: Przykładowy dokument użytkownika w Azure Cosmos DB

Pole role definiuje uprawnienia w systemie i może przyjmować wartość user (użytkownik standardowy) lub admin (administrator systemu).

4.3.2 Zadanie do wydruku (kontener jobs)

Dokument zadania (listing 2) agreguje informacje o pliku sterującym, powiązonym użytkowniku, metrykach czasu oraz bieżącym stanie realizacji procesu produkcyjnego.

```
1  {
2    "id": "<timestamp string>",
3    "fileId": "20251203_014648_model.gcode",
4    "originalFileName": "model.gcode",
5    "userId": "user_<timestamp>",
6    "userEmail": "student@uwr.edu.pl",
7    "status": "printing",
8    "scheduledAt": "2025-12-01T15:00:00Z",
9    "createdAt": "2025-11-30T10:00:00Z",
10   "startedAt": "2025-12-01T15:01:00Z",
11   "completedAt": "2025-12-01T17:30:00Z",
12   "progress": 87.5,
13   "printTime": 4500,
14   "printTimeLeft": 600,
15   "deviceId": "raspberry-pi-mkt-01",
16   "failureReason": null
17 }
```

Listing 2: Przykładowy dokument zadania do wydruku w Cosmos DB

Pole status może przyjmować wartości: pending, scheduled, waiting_for_printer_ready, delayed, printing, completed, failed, cancelled.

4.3.3 Token odświeżania (kontener refresh-tokens)

W celu zapewnienia bezpiecznej, bezstanowej autoryzacji sesji, tokeny odświeżania (ang. *refresh tokens*) są utrwalane w dedykowanym kontenerze (listing 3).

```
1  {
2    "id": "<userId>_<timestamp>",
3    "userId": "user_<timestamp>",
4    "token": "<JWT string>",
5    "expiresAt": "2025-12-08T00:00:00Z",
6    "createdAt": "2025-12-01T00:00:00Z"
7  }
```

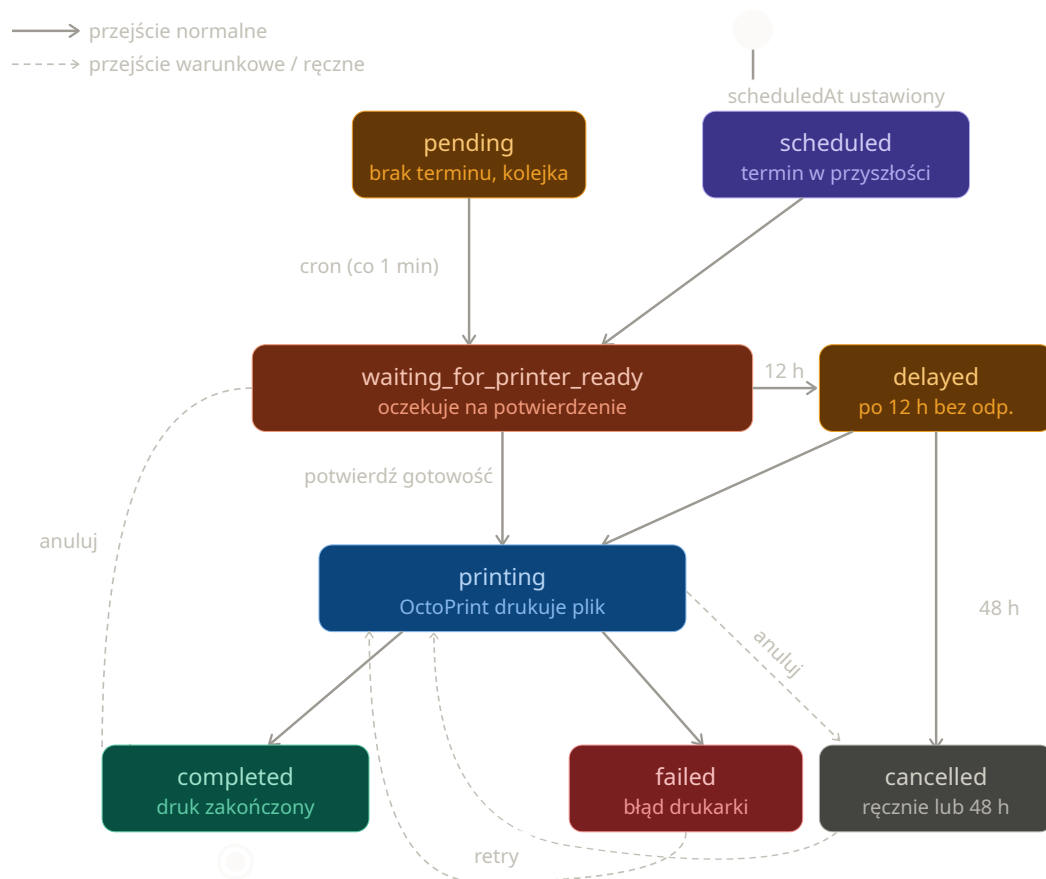
Listing 3: Przykładowy dokument refresh tokena

4.4 Cykl życia zadania do wydruku

Zadanie w procesie przetwarzania i realizacji może znajdować się w jednym z następujących stanów:

- **pending** — zadanie bez określonego terminu realizacji, oczekujące w kolejce ogólnej systemu.
- **scheduled** — zadanie posiadające ściśle zaplanowany termin realizacji, oczekujące na osiągnięcie wskazanego czasu rozpoczęcia. Po nadejściu zaplanowanego czasu zadanie to otrzymuje najwyższy priorytet wykonania.
- **waiting_for_printer_ready** — usługa harmonogramująca (ang. *scheduler*) wybrała zadanie do realizacji; system oczekuje na manualne potwierdzenie gotowości urządzenia przez użytkownika.
- **delayed** — stan opóźnienia, w którym system oczekuje na potwierdzenie gotowości drukarki dłużej niż 3 h; do użytkownika wysyłane jest automatyczne powiadomienie przypominające.
- **printing** — proces drukowania modelu sterującego jest aktywnie realizowany przez oprogramowanie *OctoPrint*.
- **completed** — proces drukowania 3D został pomyślnie sfinalizowany.
- **failed** — w trakcie realizacji procesu produkcyjnego wystąpił błąd krytyczny uniemożliwiający dalsze drukowanie.
- **cancelled** — zadanie zostało anulowane ręcznie przez użytkownika bądź administratora lub automatycznie przez system po upływie 48 h oczekiwania na potwierdzenie gotowości urządzenia.

Na rysunku 4 przedstawiono formalny model maszyny stanów, obrazujący dopuszczalne przejścia fazowe w cyklu życia zadania produkcyjnego.



Rysunek 4: Diagram maszyny stanów cyklu życia zadania do wydruku.

4.5 Bezpieczeństwo i uwierzytelnianie

W celu zapewnienia poufności danych oraz kontroli dostępu, w systemie zaimplementowano dwutokenowy model uwierzytelniania oparty na standardzie JWT (*JSON Web Token*) [1]. Mechanizm ten wykorzystuje dwa rodzaje obiektów:

- **Access token** — token dostępu o krótkim czasie autoryzacji (ważny przez 15 minut), przekazywany w nagłówku protokołu HTTP (`Authorization: Bearer <token>`). Integralność i autentyczność tokenu są weryfikowane kryptograficznie przy użyciu klucza symetrycznego `JWT_SECRET`.
- **Refresh token** — token odświeżający o wydłużonym czasie ważności (wynoszącym 7 dni), utrwalany w bazie danych *Azure Cosmos DB* oraz przechowywany po stronie aplikacji klienckiej. Podpisywany jest przy użyciu dedykowanego klucza `JWT_REFRESH_SECRET`. Służy do generowania nowej pary tokenów bez konieczności ponownego wprowadzania danych uwierzytelniających przez użytkownika.

Hasła użytkowników nie są przechowywane w bazie danych w postaci jawnej. Do ich zabezpieczenia wykorzystano algorytm funkcjonału skrótu *bcrypt* z parametrem kosztu (ang. *salt rounds*) ustawionym na wartość 10.

Proces rejestracji nowych kont został ze względów bezpieczeństwa ograniczony wyłącznie do adresów poczty elektronicznej w domenie akademickiej @uwr.edu.pl. Pełna aktywacja profilu wymaga weryfikacji dwuetapowej poprzez kliknięcie w unikalny link aktywacyjny przesłany na adres e-mail użytkownika.

Kontrola dostępu na poziomie kodu realizowana jest za pomocą oprogramowania pośredniczącego (ang. *middleware*):

- `requireAuth` — odpowiada za ekstrakcję i weryfikację tokenu dostępu poprzez wysłanie wewnętrznego żądania weryfikacyjnego do punktu końcowego POST `/auth/verify` usługi *Auth Service*.
- `requireAdmin` — realizuje mechanizm kontroli dostępu opartej na rolach (ang. *Role-Based Access Control*, RBAC), sprawdzając, czy zdekodowany profil użytkownika posiada przypisane uprawnienia klasy admin.

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

Usługa *Auth Service* odpowiada za procesy uwierzytelniania, autoryzacji oraz zarządzania kontami użytkowników. W zakres jej funkcjonalności wchodzi: rejestracja, logowanie i wylogowanie użytkowników, odświeżanie tokenów sesyjnych oraz weryfikacja adresów poczty elektronicznej.

Komponent ten został zaimplementowany w środowisku uruchomieniowym Node.js z wykorzystaniem frameworka Express.js oraz języka TypeScript. Serwis realizuje operacje wejścia-wyjścia poprzez komunikację z bazą danych *Azure Cosmos DB* (w ramach kontenerów *users* oraz *refresh-tokens*). Ponadto integruje się z zewnętrznym serwerem SMTP firmy Google w celu dystrybucji wiadomości aktywacyjnych, wykorzystując do tego celu bibliotekę *Nodemailer*.

5.1.2 Punkty końcowe API

W tabeli 3 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez usługę *Auth Service*.

Tabela 3: Punkty końcowe usługi Auth Service.

Metoda	Ścieżka	Opis
POST	/auth/register	Rejestracja nowego użytkownika, walidacja domeny e-mail, generowanie szyfru hasła oraz wysłanie wiadomości z linkiem aktywacyjnym.
POST	/auth/login	Uwierzytelnienie użytkownika; generowanie i zwrot pary tokenów: <i>access token</i> oraz <i>refresh token</i> .
POST	/auth/logout	Unieważnienie tokenu odświeżającego i zakończenie aktywnej sesji użytkownika.

Tabela 3 – Kontynuacja z poprzedniej strony

Metoda	Ścieżka	Opis
PATCH	/auth/refresh	Generowanie nowego tokenu dostępu na podstawie ważnego tokenu odświeżającego.
POST	/auth/verify	Weryfikacja poprawności tokenu dostępu, wykorzystywana wewnętrznie przez pozostałe mikroserwisy.
GET	/auth/verify-email	Aktywacja konta użytkownika po odebraniu i weryfikacji tokenu z linku aktywacyjnego.
POST	/auth/resend-verification	Ponowne wygenerowanie i wysłanie wiadomości weryfikacyjnej na adres e-mail.
GET	/health	Punkt końcowy diagnostyki stanu działania usługi (ang. <i>health check</i>).

5.1.3 Kluczowe fragmenty implementacji

Na listingu 4 przedstawiono implementację funkcji odpowiedzialnych za generowanie tokenu dostępu oraz tokenu odświeżającego przy użyciu biblioteki jsonwebtoken [1].

```

1 export function generateAccessToken(user: User): string {
2   const payload: JWTPayload = {
3     userId: user.id,
4     email: user.email,
5     role: user.role
6   };
7   return jwt.sign(payload, CONFIG.JWT_SECRET, { expiresIn: '15m',
8     });
9 }
10 export function generateRefreshToken(user: User): string {
11   return jwt.sign(
12     { userId: user.id },

```

```

13 CONFIG.JWT_REFRESH_SECRET,
14 { expiresIn: '7d' }
15 );
16 }

```

Listing 4: Implementacja generowania tokenów JWT (token-service.ts)

Proces walidacji tokenu odświeżającego (ang. *refresh token*) jest dwuetapowy. Obejmuje zarówno kryptograficzną weryfikację poprawności podpisu cyfrowego struktury JWT, jak i asynchroniczne sprawdzenie obecności oraz ważności powiązanego rekordu w bazie danych *Azure Cosmos DB*. Takie podejście architektoniczne umożliwia natychmiastowe, zdalne unieważnianie sesji (ang. *token revocation*) w momencie wylogowania się użytkownika z systemu. Kod realizujący tę procedurę zaprezentowano na listingu 5.

```

1 export async function validateRefreshToken(
2   token: string
3 ): Promise<string | null> {
4   try {
5     const decoded = jwt.verify(
6       token,
7       CONFIG.JWT_REFRESH_SECRET
8     ) as { userId: string };
9
10    const query = `SELECT * FROM c
11      WHERE c.token = @token
12      AND c.userId = @userId
13      AND c.expiresAt > @now`;
14
15    const { resources } = await refreshTokensContainer.items.query(
16      {
17        query,
18        parameters: [
19          { name: '@token', value: token },
20          { name: '@userId', value: decoded.userId },
21          { name: '@now', value: new Date().toISOString() }
22        ]
23      }).fetchAll();
24
25    return resources.length > 0 ? decoded.userId : null;
26  } catch {

```

```

26     return null;
27 }
28 }

```

Listing 5: Implementacja walidacji refresh tokenu (token-service.ts)

5.1.4 Weryfikacja adresu poczty elektronicznej

Po pomyślnym zakończeniu procesu rejestracji system generuje unikalny, 64-znakowy token w formacie szesnastkowym. Jest on tworzony przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych z modułu `crypto` (`crypto.randomBytes`). Zapewnia to wysoką entropię i odporność na ataki typu *brute-force*.

Wygenerowany identyfikator jest zapisywany w bazie danych wraz z datą ważności, a do użytkownika wysyłana jest wiadomość e-mail zawierająca odsyłacz aktywacyjny skierowany do punktu końcowego systemu:

```
GET /auth/verify-email?token=<hex>
```

Po odebraniu żądania przez usługę *Auth Service*, system przeprowadza weryfikację poprawności oraz ważności tokenu. W przypadku pozytywnego rezultatu walidacji, atrybut `isVerified` w dokumencie użytkownika przyjmuje wartość `true`, a konto zostaje w pełni aktywowane. Na zakończenie procesu serwer wykonuje przekierowanie (ang. *HTTP redirect*) przeglądarki użytkownika do adresu bazowego aplikacji klienckiej.

5.2 User Service

5.2.1 Opis ogólny

Usługa *User Service* realizuje zadania związane z zarządzaniem profilami użytkowników, modyfikacją ich uprawnień oraz agregacją danych dotyczących powiązanych zadań wytwórczych. Komponent ten wykonuje operacje odczytu i zapisu danych w ramach kontenerów `users` oraz `jobs` w bazie danych *Azure Cosmos DB*.

Autoryzacja żądań oraz weryfikacja tożsamości użytkowników są realizowane w sposób scentralizowany poprzez przesyłanie wewnętrznych zapytań do punktu końcowego `POST /auth/verify` opisaną wcześniej usługi *Auth Service*.

5.2.2 Punkty końcowe API

W tabeli 4 wyszczególniono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez usługę *User Service*, z uwzględnieniem podziału na operacje standardowe oraz administracyjne.

Tabela 4: Punkty końcowe usługi User Service.

Metoda	Ścieżka	Opis
GET	/users/me	Pobranie danych profilowych zalogowanego użytkownika.
PATCH	/users/me	Aktualizacja danych osobowych użytkownika (imię oraz nazwisko).
GET	/users/me/jobs	Pobranie historii wszystkich zadań przypisanych do zalogowanego użytkownika.
GET	/users/me/stats	Pobranie statystyk użytkownika, obejmujących m.in. zagregowaną liczbę zadań z podziałem na ich statusy.
GET	/users/jobs/upcoming	Pobranie listy zadań oczekujących na realizację (posiadających status pending lub scheduled).
GET	/users/jobs/recent-completed	Pobranie historycznej listy ostatnio sfinalizowanych procesów druku.
GET	/users	Pobranie rejestru wszystkich użytkowników systemu (dostęp zastrzeżony dla administratora).

Tabela 4 – ciąg dalszy z poprzedniej strony

Metoda	Ścieżka	Opis
PATCH	/users/:id/role	Modyfikacja roli i poziomu uprawnień użytkownika o wskazanym identyfikatorze (dostęp zastrzeżony dla administratora).
DELETE	/users/:id	Trwałe usunięcie konta użytkownika na podstawie parametru identyfikatora (dostęp zastrzeżony dla administratora).
GET	/users/:id/jobs	Pobranie pełnej listy zadań powiązanych z wybranym kontem użytkownika (dostęp zastrzeżony dla administratora).

5.2.3 Mechanizm autoryzacji żądań

Na listingu 6 zaprezentowano produkcyjną implementację oprogramowania pośredniczącego `requireAuth`. Komponent ten w pełni wykorzystuje silne typowanie języka TypeScript, w tym parametry generyczne interfejsów `Request` oraz `Response` z biblioteki `express`.

Procedura autoryzacji polega na bezpiecznej ekstrakcji tokenu dostępu z nagłówka `Authorization`, a następnie wykonaniu asynchronicznego żądania sieciowego metodą `POST` do punktu końcowego usługi *Auth Service*. Po odebraniu odpowiedzi struktura danych jest rzutowana na dedykowany typ `Data`. W przypadku poprawnej walidacji następuje asercja typu obiektu żądania, wstrzyknięcie profilu użytkownika do pola `user` oraz wywołanie funkcji `next()`, która przekazuje sterowanie do kolejnego elementu w potoku przetwarzania (ang. *middleware chain*).

```

1 import { CONFIG } from "../config/config";
2 import type { Request, Response, NextFunction } from "express";
3 import { Data, AuthUser } from "../mwTypes";
4

```

```

5  const requireAuth = async (
6  req: Request<{}>, unknown, {}, {}>,
7  res: Response<{error: string}>,
8  next: NextFunction
9  ): Promise<void | {error: string}> => {
10     try {
11         const token: string | undefined = req.headers.authorization?.
            replace('Bearer ', '');
12
13         if (!token) {
14             res.status(401).json({ error: 'Missing authentication token'
15             });
16             return;
17         }
18
19         const response: globalThis.Response = await fetch(`${CONFIG.A
20         UTH_SERVICE_URL}/auth/verify`, {
21             method: 'POST',
22             headers: {
23                 'Authorization': `Bearer ${token}`,
24                 'Content-Type': 'application/json'
25             }
26         });
27
28         const data: Data = await response.json() as Data;
29
30         if (!response.ok) {
31             throw new Error('Authentication failed');
32         }
33
34         (req as any).user = data.user;
35
36         next();
37     } catch (err) {
38         res.status(401).json({ error: 'Invalid authentication token'
39         });

```

```
40 export default requireAuth;
```

Listing 6: Implementacja oprogramowania pośredniczącego `requireAuth` (`requireAuth.ts`)

5.3 Files Service

5.3.1 Opis ogólny

Usługa *Files Service* stanowi mikroserwis zaimplementowany w języku Python z wykorzystaniem mikroframeworka Flask. Komponent ten odpowiada za asynchroniczne przyjmowanie plików z kodem sterującym (G-code) od aplikacji klienckiej, ich bezpieczne utrwalanie w repozytorium obiektowym *Azure Blob Storage* oraz rozgłaszanie zdarzeń systemowych za pośrednictwem magistrali komunikatów *Azure Service Bus*.

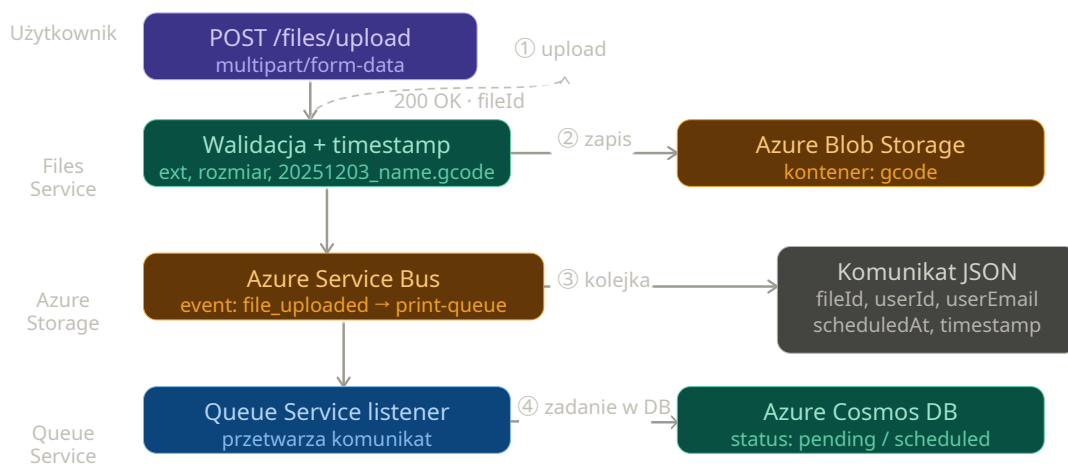
5.3.2 Procedura przetwarzania i przesyłania plików

Przebieg procesu przesyłania pliku oraz jego rejestracji w systemie realizowany jest sekwencyjnie według następującego scenariusza:

1. **Inicjacja żądania:** Aplikacja kliencka wysyła żądanie `POST /files/upload` zawierające plik zakodowany w formacie `multipart/form-data`.
2. **Weryfikacja uprawnień:** Oprogramowanie pośredniczące `require_auth` deleguje proces walidacji tokenu uwierzytelniającego do usługi *Auth Service*.
3. **Walidacja potoku wejściowego:** System weryfikuje poprawność formatu pliku (akceptowane jest wyłącznie rozszerzenie `.gcode`), jego rozmiar (wprowadzono limit do 50 MB) oraz opcjonalnie strukturę wewnętrzną (w przypadku aktywacji flagi konfiguracyjnej `STRICT_CONTENT_CHECK`).
4. **Sanitaryzacja i unikalność nazw:** Nazwa pliku wejściowego jest modyfikowana poprzez dodanie prefiksu czasowego zawierającego kompletną datę i godzinę odebrania żądania (np. `20251203_014648_model.gcode`), co zapobiega konfliktom nadpisywania zasobów.
5. **Utrwalenie zasobu:** Plik w formacie binarnym zostaje przesłany i zapisany w dedykowanym kontenerze `gcode` w ramach usługi *Azure Blob Storage*.

6. **Publikacja zdarzenia:** Do kolejki print-queue magistrali *Azure Service Bus* wysyłany jest komunikat w formacie JSON, który inicjuje dalsze etapy cyklu życia zadania.

Na rysunku 5 zobrazowano schematyczny przepływ danych podczas procedury przesyłania i kolejkowania pliku G-code.



Rysunek 5: Przepływ przesyłania pliku G-code — od zapytania HTTP do integracji z kolejką.

Na listingu 7 zaprezentowano implementację końcowego etapu obsługi żądania w ramach kontrolera usługi *Files Service*. Po pomyślnej walidacji i zapisie pliku w repozytorium obiekowym, struktura danych reprezentująca zdarzenie jest serializowana do formatu JSON i przekazywana do metody nadawczej klienta *Azure Service Bus*. Całość operacji została otoczona blokiem obsługi wyjątków (try-except), co gwarantuje stabilność działania mikroserwisu i zwrócenie ustrukturyzowanej odpowiedzi o błędzie w formacie JSON wraz ze statusem HTTP 500 w przypadku awarii infrastruktury chmuwej.

```
1 message = {
2     'event': 'file_uploaded',
3     'fileId': filename,
4     'originalFileName': original_filename,
5     'userId': user_id,
6     'userEmail': user_email,
```

```

7         'timestamp': str(time.time()),
8         'scheduledAt': request.form.get('scheduledAt')
9     }
10
11     sb_client = current_app.config.get('SERVICE_BUS_CLIENT')
12     if sb_client:
13         with sb_client:
14             sender = sb_client.get_queue_sender(queue_name='print-
15 queue')
16             sender.send_messages([{'body': json.dumps(message)}])
17
18         return jsonify({'status': 'success', 'fileId': filename}), 200
19
20 except Exception as e:
21     return jsonify({'error': str(e)}), 500

```

Listing 7: Implementacja publikacji komunikatu w Azure Service Bus (files-controller.py)

5.4 Queue Service

5.4.1 Opis ogólny

Usługa *Queue Service* to mikroservis zaimplementowany w środowisku uruchomieniowym Node.js przy użyciu języka TypeScript oraz standardu modułów ECMAScript (ang. *ECMAScript Modules*, ESM). Głównym zadaniem komponentu jest asynchroniczne i bezprzerwowe nasłuchiwanie (ang. *listening*) komunikatów publikowanych w magistrali *Azure Service Bus*, a następnie ich transformacja i utrwalanie w postaci dokumentów zadań produkcyjnych w bazie danych *Azure Cosmos DB*.

Dodatkowo serwis udostępnia dedykowany interfejs programistyczny HTTP API, który umożliwia innym komponentom systemu (w tym aplikacji klienckiej) monitorowanie stanu kolejki oraz zarządzanie priorytetami zadań.

5.4.2 Mechanizm nasłuchiwania kolejki

Na listingu 8 przedstawiono implementację procedury odpowiedzialnej za rejestrację i obsługę potoku zdarzeń pochodzących z kolejki chmurowej. Logika biznesowa reali-

zuje bezpieczne parsowanie komunikatów w formacie JSON oraz obsługuje scenariusz awaryjny (ang. *dead-lettering/abandonment*) poprzez wywołanie metody `abandonMessage()`, co pozwala na zachowanie spójności danych w przypadku wystąpienia błędów infrastrukturalnych.

```
1 export function startPrintQueueListener(container, receiver) {
2   if (!container) throw new Error('container is required for
   printQueueListener');
3   if (!receiver) throw new Error('receiver is required for
   printQueueListener');
4
5   const messageHandler = async (message) => {
6     try {
7       let data = message.body;
8       if (typeof data === 'string') {
9         try {
10          data = JSON.parse(data);
11        } catch(e) {
12          await receiver.abandonMessage(message);
13          return;
14        }
15      }
16
17      if (!data || data.event !== 'file_uploaded') {
18        return;
19      }
20
21      const job = {
22        id: Date.now().toString(),
23        fileId: data.fileId,
24        originalFileName: data.originalFileName || null,
25        userId: data.userId,
26        userEmail: data.userEmail,
27        status: data.scheduledAt ? 'scheduled' : 'pending',
28        scheduledAt: data.scheduledAt || null,
29        createdAt: getLocalISO(),
30      };
31
32      try {
```

```

33     await container.items.upsert(job);
34   } catch(upErr) {
35     throw upErr;
36   }
37 } catch(err) {
38   try {
39     await receiver.abandonMessage(message);
40   } catch(e) {
41     // Obsługa błędu komunikacji z repozytorium kolejki
42   }
43 }
44 };
45
46 const errorHandler = (error) => {
47   console.error('EVENT ERROR:', error);
48 };
49
50 receiver.subscribe({
51   processMessage: messageHandler,
52   processError: errorHandler
53 });
54 }

```

Listing 8: Implementacja mechanizmu nasłuchiwania kolejki (printQueueListener.js)

5.4.3 Punkty końcowe HTTP API

W tabeli 5 zestawiono punkty końcowe udostępniane przez interfejs sieciowy usługi *Queue Service*.

5.5 Printer Service

5.5.1 Opis ogólny

Usługa *Printer Service* stanowi centralny komponent orkiestracji systemu, odpowiedzialny za zaawansowane harmonogramowanie, weryfikację stanów urządzeń oraz nadzór nad realizacją procesów wytwórczych. Mikroserwis ten został zaimplementowany w języku TypeScript i składa się z trzech autonomicznych podsystemów logicznych:

Tabela 5: Punkty końcowe usługi Queue Service.

Metoda	Ścieżka	Opis
GET	/queue	Pobranie spaginowanej listy zadań z możliwością definiowania filtrów wyszukiwania.
GET	/queue/next	Pobranie kolejnego zadania przeznaczonego bezpośrednio do realizacji (zwraca kod 204 No Content w przypadku braku pozycji).
PATCH	/queue/cancel/:id	Anulowanie wybranego zadania drukowania (dostęp ograniczony do roli administratora).
GET	/queues	Pobranie metryk i informacji o stanie kolejek w ramach usługi <i>Azure Service Bus</i> (dostęp dla administratora).
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

- **Harmonogram zadań** (uruchamiany cyklicznie z interwałem jednogodzinowym) — odpowiada za sekwencyjny wybór kolejnego zadania produkcyjnego przeznaczonego do realizacji na podstawie kryteriów czasowych.
- **Monitor gotowości urządzenia** (uruchamiany cyklicznie z interwałem pięciominutowym) — nadzoruje i weryfikuje statusy zadań oczekujących na manualne potwierdzenie gotowości drukarki, egzekwując zdefiniowane limity czasowe (ang. *timeouts*).
- **Odbiornik telemetrii** (ang. *Telemetry Listener*) — asynchroniczny komponent zintegrowany z usługą *Azure Event Hubs*, którego zadaniem jest ciągły odbiór zdarzeń telemetrycznych transmitowanych przez agenta sprzętowego oraz bieżąca aktualizacja stanów zadań w bazie danych.

5.5.2 Mechanizm harmonogramowania

W ramach każdej minutowej iteracji usługa harmonogramująca wykonuje sekwencję operacji kontrolno-decyzyjnych:

1. **Weryfikacja aktywnego procesu:** System sprawdza, czy w bazie danych istnieje

zadanie o statusie printing. W przypadku wykrycia aktywnego procesu drukowania, bieżąca iteracja harmonogramu zostaje natychmiast przerwana.

2. **Weryfikacja stanu oczekiwania:** System kontroluje, czy proces nie został wstrzymany przez zadanie oczekujące na potwierdzenie fizycznej gotowości drukarki. Jeżeli takie zadanie istnieje, wybór kolejnej pozycji z kolejki zostaje pominięty.
3. **Wybór i alokacja zasobu:** W przypadku braku blokad, system wyszukuje zadanie o statusie pending lub scheduled o najwcześniejszej sygnaturze czasowej (scheduledAt), a następnie dokonuje atomowej zmiany jego statusu na waiting_for_printer_ready.

Na listingu 9 zaprezentowano implementację logiki wyboru oraz rezerwacji zadania, z uwzględnieniem dialektu zapytań SQL specyficznego dla bazy danych *Azure Cosmos DB*.

```
1  const query = '  
2    SELECT TOP 1 * FROM c  
3    WHERE (c.status = 'scheduled' OR c.status = 'pending')  
4    AND (c.scheduledAt <= "${now}"  
5    OR IS_NULL(c.scheduledAt)  
6    OR NOT IS_DEFINED(c.scheduledAt))  
7    ORDER BY c.scheduledAt ASC  
8  ';  
9  const { resources: jobs } = await container.items.query(query).  
    fetchAll();  
10  
11  for (const job of jobs) {  
12    if(!job.id) continue;  
13  
14    job.status = 'waiting_for_printer_ready';  
15    job.waitingStartedAt = nowDate;  
16  
17    await container.item(job.id, job.id).replace(job);  
18  }
```

Listing 9: Implementacja mechanizmu wyboru i rezerwacji zadań z kolejki (startScheduledJobs.ts)

5.5.3 Monitor gotowości urządzenia

Monitor gotowości odpowiada za stałe nadzorowanie zadań znajdujących się w stanach `waiting_for_printer_ready` oraz `delayed`. Komponent ten realizuje wielopoziomowy, deterministyczny proces przypomnień i eskalacji stanów:

- **Po 60 minutach** — następuje wyzwolenie i wysłanie pierwszego powiadomienia przypominającego.
- **Po 3 godzinach** — następuje wysłanie drugiego powiadomienia przypominającego.
- **Po 12 godzinach** — system dokonuje automatycznej zmiany statusu zadania na `delayed`.
- **Po 48 godzinach** — proces oczekiwania zostaje przerwany, a zadanie otrzymuje status terminalny `cancelled`.

Dystrybucja komunikatów alarmowych i powiadomień realizowana jest za pośrednictwem uniwersalnego mechanizmu sieciowego typu `Webhook`, którego adres docelowy jest definiowany w konfiguracji systemu poprzez zmienną środowiskową `ALERT_WEBHOOK_URL`.

5.5.4 Odbiornik telemetrii

Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, który jest natywnie udostępniany przez usługę *Azure IoT Hub*. W tabeli 6 zestawiono typy zdarzeń telemetrycznych generowanych przez agenta sprzętowego oraz powiązane z nimi akcje modyfikacji dokumentów w bazie danych *Azure Cosmos DB*.

Tabela 6: Zdarzenia telemetryczne przetwarzane przez usługę Printer Service.

Zdarzenie	Akcja w bazie danych Azure Cosmos DB
print_started	Zmiana statusu zadania na printing oraz rejestracja sygnatury czasowej w polu startedAt.
print_progress	Sukcesywna aktualizacja metryk postępu w polach progress, printTime oraz printTimeLeft.
print_completed	Zmiana statusu na completed wraz z zapisem czasu zakończenia (completedAt) i całkowitego czasu trwania procesu (printDuration).
print_failed	Zmiana statusu na failed oraz utrwalenie kodu błędu lub przyczyny awarii w polu failureReason.
print_cancelled	Zmiana statusu na cancelled oraz rejestracja momentu przerwania pracy w polu cancelledAt.

5.5.5 Punkty końcowe HTTP API

W tabeli 7 zestawiono i opisano punkty końcowe (ang. *endpoints*) wystawione przez interfejs sieciowy usługi *Printer Service*.

Tabela 7: Punkty końcowe usługi Printer Service.

Metoda	Ścieżka	Opis
GET	/printer/jobs	Pobranie spaginowanej listy zadań z obsługą dynamicznego filtrowania wyników.
GET	/printer/jobs/:id	Pobranie szczegółowych parametrów i metryk wybranego zadania.
GET	/printer/jobs/:id/progress	Pobranie bieżących informacji o postępie i szacowanym czasie zakończenia aktywnego procesu druku.

Tabela 7 – ciąg dalszy z poprzedniej strony

Metoda	Ścieżka	Opis
POST	/printer/jobs/:id/cancel	Wymuszenie przerwania i anulowania zadania drukowania (dostęp zastrzeżony dla administratora).
POST	/printer/jobs/:id/retry	Ponowne dodanie wybranego (np. przerwane) zadania do kolejki produkcyjnej.
POST	/printer/jobs/:id/confirm-ready	Zdalne potwierdzenie fizycznej gotowości urządzenia i stołu roboczego przez użytkownika.
DELETE	/printer/jobs/:id	Trwałe usunięcie rekordu zadania z bazy danych (dostęp zastrzeżony dla administratora).
GET	/printer/status	Pobranie aktualnego stanu operacyjnego fizycznej drukarki 3D.
GET	/printer/metrics	Pobranie zagregowanych metryk statystycznych systemowych zadań z podziałem na statusy.
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

5.6 Video Service

5.6.1 Opis ogólny

Usługa *Video Service* to mikroserwis zaimplementowany w środowisku uruchomieniowym Node.js z wykorzystaniem frameworka Express.js oraz języka JavaScript. Komponent ten odpowiada za tymczasowe przechowywanie aktualnego adresu URL strumienia wideo z kamery systemu *OctoPrint* w pamięci operacyjnej serwera (ang. *in-memory storage*), co eliminuje konieczność ciągłego odpytywania bazy danych o wolnozmiennie parametry sieciowe.

Agent uruchamiany na mikrokomputerze Raspberry Pi rejestruje bieżący adres kamery (lokalny adres IP lub adres publiczny udostępniany za pośrednictwem tunelu sieciowego *Cloudflare*) po każdym zainicjowaniu systemu operacyjnego. Aplikacja kliencka (frontendowa) cyklicznie pobiera ten adres i renderuje dynamiczny strumień wideo bezpośrednio w interfejsie użytkownika w elemencie `<img>` z wykorzystaniem formatu MJPEG (*Motion JPEG*).

5.6.2 Punkty końcowe API

W tabeli 8 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez interfejs sieciowy usługi *Video Service*.

Tabela 8: Punkty końcowe usługi Video Service.

Metoda	Ścieżka	Opis
POST	/camera-url	Rejestracja lub aktualizacja nowego adresu URL strumienia wideo z kamery.
GET	/camera-url	Pobranie aktualnego adresu URL kamery w celu poprawnego wyrenderowania podglądu.
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

6 Implementacja warstwy prezentacji

6.1 Struktura architektoniczna aplikacji klienckiej

Aplikacja frontendowa została zaimplementowana z wykorzystaniem biblioteki *React* w wersji 18 oraz języka TypeScript. Jako środowisko deweloperskie i narzędzie budujące (ang. *bundler*) wykorzystano *Vite*, natomiast zarządzanie mapowaniem adresów URL i nawigacją realizowane jest przy użyciu biblioteki *React Router* w wersji 6.

Zarządzanie globalnym stanem aplikacji klienckiej (obejmującym dane profilowe zalogowanego użytkownika, bieżący status operacyjny drukarki oraz metryki systemowe) powierzono bibliotece *Zustand*. W celu zachowania ciągłości sesji i preferencji interfejsu (takich jak motyw graficzny czy wybrany język), wykorzystano oprogramowanie pośredniczące *persist*, które automatyzuje proces synchronizacji i trwałego przechowywania danych w pamięci podręcznej *localStorage* przeglądarki internetowej.

Struktura drzewa katalogów aplikacji została zorganizowana w sposób modułowy i przedstawia się następująco:

- `src/components/` — komponenty interfejsu użytkownika wielokrotnego użytku, odpowiadające za renderowanie pojedynczych elementów (m.in. `Card`, `StatusBadge`, `ProgressBar`, `ConfirmDialog`, `Layout`, `LoadingSpinner`),
- `src/pages/` — komponenty wysokiego poziomu reprezentujące autonomiczne widoki aplikacji (m.in. `HomePage`, `LoginPage`, `RegisterPage`, `DashboardPage`, `UploadPage`, `ProfilePage`, `PrintControlPage`, `AdminDashboard`),
- `src/services/api.ts` — implementacja scentralizowanego klienta API odpowiedzialnego za komunikację siecią z backendową warstwą mikroserwisów,
- `src/store/useStore.ts` — implementacja globalnego magazynu stanu (ang. *store*) biblioteki *Zustand*,
- `src/i18n/` — konfiguracja internacjonalizacji aplikacji oparta na bibliotece *i18next* wraz z plikami translacji interfejsu,
- `src/types/` — definicje interfejsów i typów języka TypeScript, w tym struktur `User`, `Job` oraz `PrinterStatus`,

- `src/utils/formatters.ts` — funkcje pomocnicze realizujące operacje bezstanowe, takie jak formatowanie dat, rozmiarów plików czy czasu.

6.2 Scentralizowany klient API

Komunikacja z rozproszoną warstwą backendową realizowana jest przez klasę `ApiClient` zdefiniowaną w module `api.ts`. Klasa ta enkapsuluje i konfiguruje niezależne instancje biblioteki *Axios* dedykowane dla poszczególnych mikroservisów systemu (*AuthService*, *User Service*, *Printer Service* oraz *Files Service*).

W celu automatyzacji obsługi protokołu HTTP, w konfiguracji instancji zaimplementowano mechanizmy przechwytyjące (ang. *interceptors*). Odpowiadają one za automatyczne wstrzykiwanie tokenu dostępu do nagłówka *Authorization* (w standardzie *Bearer*) dla każdego żądania wychodzącego, a także realizują asynchroniczną procedurę odświeżania sesji w przypadku przechwycenia odpowiedzi z kodem błędu HTTP 401 (*Unauthorized*).

Na listingu 10 przedstawiono implementację interceptora odpowiedzi, który w sposób reaktywny zarządza odświeżaniem tokenów i ponawianiem pierwotnych żądań sieciowych.

```
1 client.interceptors.response.use(  
2   response => response,  
3   async (error: AxiosError) => {  
4     if (error.response?.status === 401) {  
5       try {  
6         const refreshToken = localStorage.getItem('refreshToken');  
7         if (refreshToken) {  
8           const { data } =  
9             await this.authClient.patch<  
10               Omit<AuthResponse, 'user'>  
11               >(  
12                 '/auth/refresh',  
13                 { refreshToken }  
14               );  
15             localStorage.setItem('accessToken', data.accessToken);  
16             localStorage.setItem('refreshToken', data.refreshToken);  
17             if (error.config) {  
18               error.config.headers.Authorization = `Bearer ${data.`
```

```

    accessToken}'`;
19         return axios.request(error.config);
20     }
21 }
22 } catch {
23     this.logout();
24 }
25 }
26 return Promise.reject(error);
27 }
28 );

```

Listing 10: Implementacja interceptora automatycznego odświeżania tokenów JWT (api.ts)

6.3 Mechanizm routingu i ochrona tras

W celu zapewnienia selektywnej kontroli dostępu do zasobów warstwy prezentacji, w aplikacji zaimplementowano mechanizm tras chronionych (ang. *protected routes*). Wykorzystuje on autorski komponent opakowujący, który dynamicznie weryfikuje stan sesji użytkownika. Dostęp do widoków domenowych wymaga aktywnego uwierzytelnienia, podczas gdy trasy administracyjne realizują dodatkową autoryzację opartą na rolach (RBAC), sprawdzając uprawnienia użytkownika przed wyrenderowaniem komponentów docelowych.

Na listingu 11 zaprezentowano deklaratywną implementację komponentu `ProtecteatedRoute`, zintegrowanego z globalnym magazynem stanu systemu.

```

1 function ProtectedRoute({ children, requireAdmin = false }: {
    children: React.ReactNode; requireAdmin?: boolean }) {
2
3     const { isAuthenticated, user } = useStore();
4
5     if (!isAuthenticated) {
6         return <Navigate to="/login" replace />;
7     }
8
9     if (requireAdmin && user?.role !== 'admin') {
10        return <Navigate to="/" replace />;

```



```

11   }
12
13   return <>{children}</>;
14 }

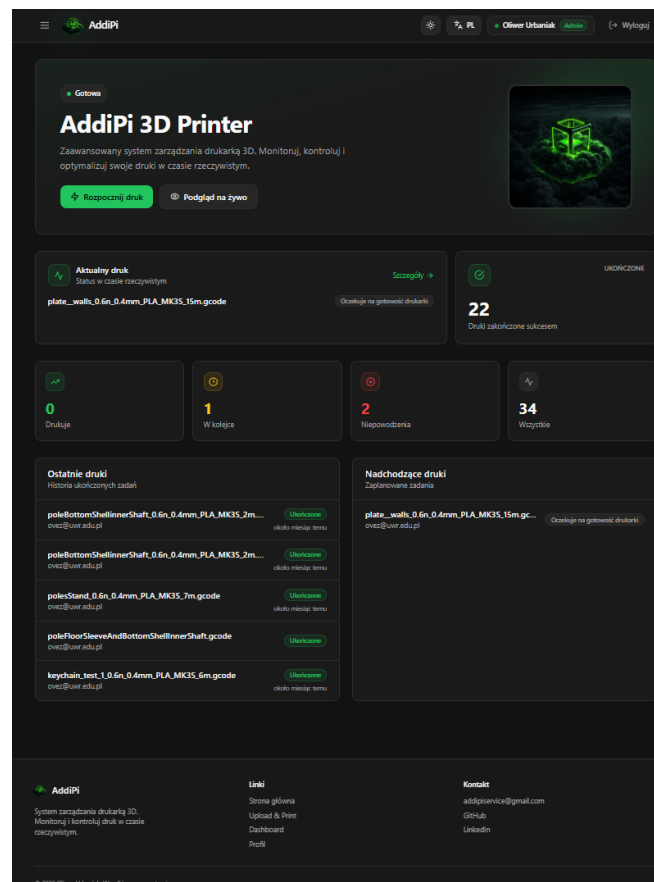
```

Listing 11: Implementacja komponentu tras chronionych (App.tsx)

6.4 Ekran aplikacji klienckiej

6.4.1 Strona główna systemu

Strona główna (ang. *landing page*) agreguje i prezentuje kluczowe parametry operacyjne całego ekosystemu. Na widoku tym wyświetlany jest aktualny status fizycznej drukarki 3D, postęp procentowy i czasowy realizacji bieżącego procesu produkcyjnego, rejestr ostatnio ukończonych wydruków oraz lista nadchodzących zadań zaplanowanych w kolejce. W celu zapewnienia wysokiej reaktywności, dane prezentowane w interfejsie są cyklicznie odświeżane z interwałem 5 sekund. Graficzną reprezentację tego ekranu przedstawiono na rysunku 6.



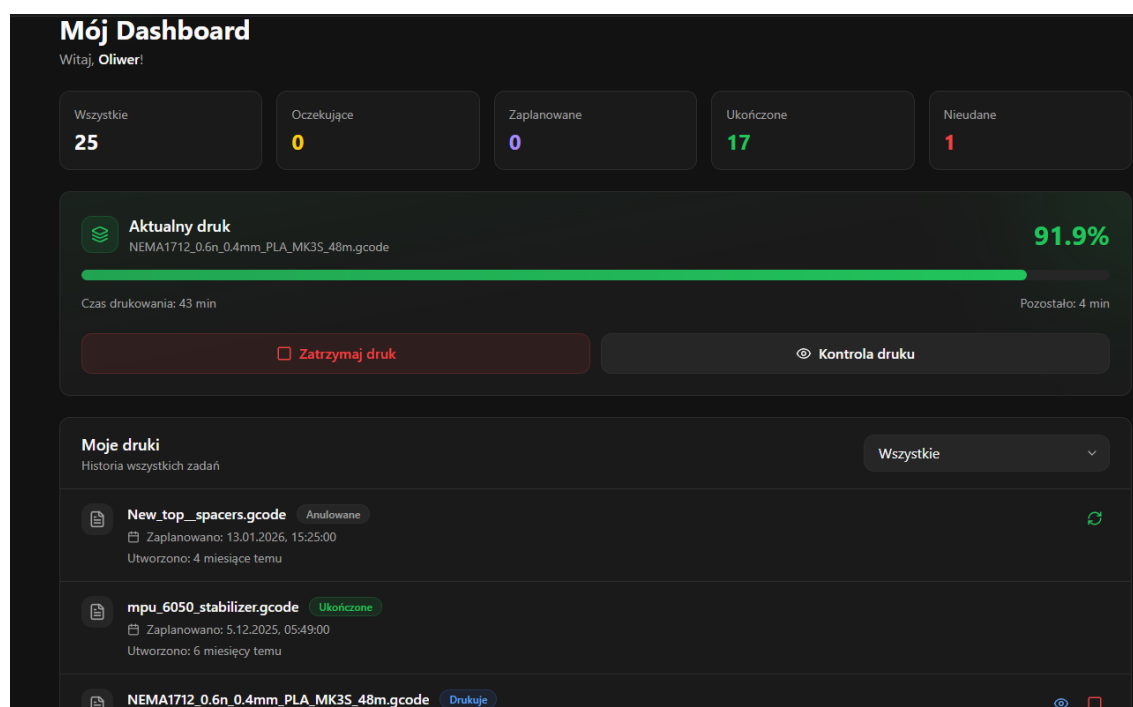
Rysunek 6: Widok strony głównej portalu AddiPi.

6.4.2 Panel użytkownika

Panel użytkownika (ang. *User Dashboard*) stanowi spersonalizowane centrum dowodzenia procesami wytwórczymi przypisanymi do danego konta. Widok ten zaprezentowano na rysunku 7. Ekran ten udostępnia:

- zagregowane statystyki zadań (liczbę przesłanych zleceń z podziałem na statusy),
- parametry aktywnego procesu drukowania, obejmujące dynamiczny pasek postępu oraz estymowany czas pozostały do zakończenia operacji,
- kompletną historię druków z modułem wyszukiwania i filtrowania danych.

Z poziomu tego panelu zalogowany operator posiada uprawnienia do manualnego anulowania aktywnego zadania, ponownego zakolejkowania procesu zakończenia niepowodzeniem (ang. *retry flow*) oraz bezpośredniego przejścia do dedykowanego ekranu szczegółowej kontroli procesu drukowania.



Rysunek 7: Widok panelu użytkownika portalu *AddiPi*.

6.4.3 Przesyłanie i planowanie zadań

Ekran przesyłania plików (*UploadPage*), zaprezentowany na rysunku 8, udostępnia użytkownikom intuicyjny interfejs typu *drag-and-drop* (przeciągnij i upuść). Komponent ten

realizuje wstępną walidację po stronie klienta, weryfikując poprawne rozszerzenie (.gcode) oraz dopuszczalny rozmiar przesyłanego wolumenu.

Widok został wyposażony w opcjonalny moduł harmonogramowania terminu realizacji zadania, bazujący na selektorze daty i godziny (ang. *date-time picker*). Interfejs w sposób reaktywny informuje operatora o statusie operacji sieciowej, wyświetlając dedykowane komunikaty o poprawnym przetworzeniu lub błędzie zapytania HTTP.

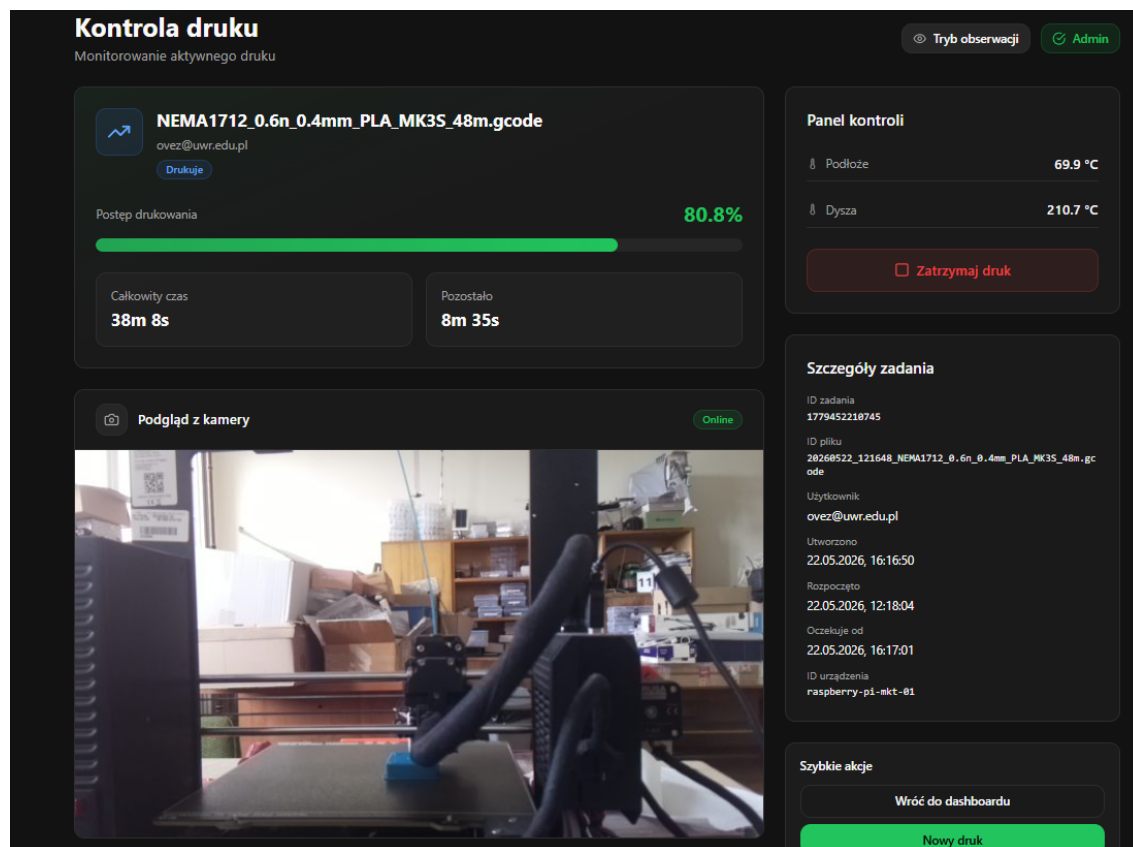
Rysunek 8: Widok panelu przesyłania i planowania zadań w portalu *AddiPi*.

6.4.4 Kontrola procesu drukowania

Dedykowany ekran kontroli procesu drukowania (*PrintControlPage*), zaprezentowany na rysunku 9, agreguje szczegółowe parametry techniczne aktywnego zlecenia. Widok ten udostępnia dynamiczny pasek postępu, bieżące odczyty temperatury dyszy ekstrudera oraz stołu roboczego, a także strumień wideo z kamery systemu *OctoPrint*. Strumień ten, przesyłany asynchronicznie w formacie MJPEG, jest renderowany bezpośrednio w strukturze dokumentu w elemencie `<img>`.

Interfejs został wyposażony w panel akcji operacyjnych, który umożliwia uprawnionym użytkownikom wstrzymanie lub wznowienie procesu wytwórczego oraz zdalne po-

twierdzenie fizycznej gotowości urządzenia do pracy. Poniżej sekcji wizualnej prezentowane są metryki i parametry konfiguracyjne powiązane z realizowanym zadaniem.



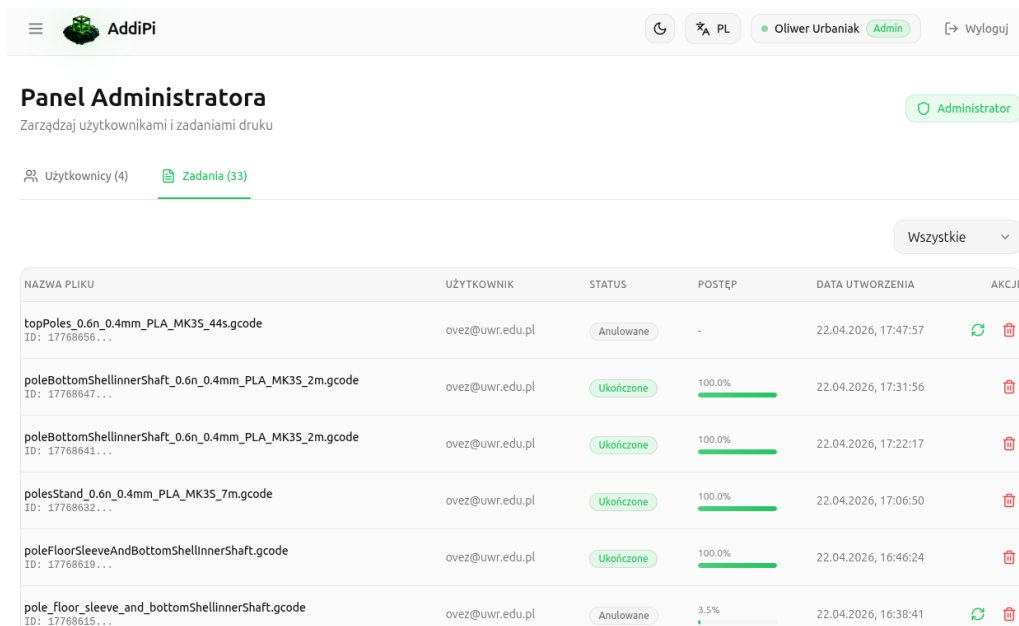
Rysunek 9: Widok panelu kontroli procesu drukowania w portalu *AddiPi*.

6.4.5 Panel administracyjny

Panel administracyjny (ang. *Admin Dashboard*) udostępnia personelowi zarządzającemu globalny wgląd w rejestry wszystkich kont użytkowników oraz zadań produkcyjnych w systemie. Dane te są prezentowane w postaci ustrukturyzowanych tabel zaimplementowanych z modułami zaawansowanego sortowania, wyszukiwania i filtrowania, co zobrazowano na rysunku 10.

Z poziomu tego widoku administrator posiada uprawnienia do:

- modyfikacji uprawnień i ról użytkowników (nadawanie statusu admin lub user),
- trwałego usuwania profili użytkowników z bazy danych,
- kompleksowego zarządzania cyklem życia wszystkich zadań w kolejce, włączając w to ich wymuszone anulowanie, ponowne zakolejkowanie lub usunięcie z rejestru.



The screenshot shows the 'Panel Administratora' (Administrator Panel) of the AddiPi system. The header includes the AddiPi logo, a user profile for 'Oliver Urbaniak' with an 'Admin' role, and a 'Wyloguj' (Logout) button. Below the header, there's a section for 'Zadania (33)' (Tasks (33)). A table lists several tasks with columns for 'NAZWA PLIKU' (File Name), 'UŻYTKOWNIK' (User), 'STATUS' (Status), 'POSTĘP' (Progress), 'DATA UTWORZENIA' (Creation Date), and 'AKCJE' (Actions). The tasks are as follows:

NAZWA PLIKU	UŻYTKOWNIK	STATUS	POSTĘP	DATA UTWORZENIA	AKCJE
topPoles_0.6n_0.4mm_PLA_MK35_44s.gcode ID: 17768656...	ovez@uwr.edu.pl	Anulowane	-	22.04.2026, 17:47:57	[Refresh] [Delete]
poleBottomShellInnerShaft_0.6n_0.4mm_PLA_MK35_2m.gcode ID: 17768647...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:31:56	[Delete]
poleBottomShellInnerShaft_0.6n_0.4mm_PLA_MK35_2m.gcode ID: 17768641...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:22:17	[Delete]
polesStand_0.6n_0.4mm_PLA_MK35_7m.gcode ID: 17768632...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:06:50	[Delete]
poleFloorSleeveAndBottomShellInnerShaft.gcode ID: 17768619...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 16:46:24	[Delete]
pole_floor_sleeve_and_bottomShellInnerShaft.gcode ID: 17768615...	ovez@uwr.edu.pl	Anulowane	3.5%	22.04.2026, 16:38:41	[Refresh] [Delete]

Rysunek 10: Widok panelu administracyjnego portalu *AddiPi*.

6.5 Internacjonalizacja aplikacji

W celu zapewnienia uniwersalności oraz dostępności systemu, interfejs użytkownika został przygotowany w dwóch wersjach językowych: polskiej (ustawionej jako domyślna) oraz angielskiej. Warstwa translacji opiera się na bibliotece *i18next*, a same tłumaczenia poszczególnych fraz i etykiet są przechowywane w ustrukturyzowanych plikach formatu JSON (*pl.json* oraz *en.json*), zlokalizowanych w dedykowanym katalogu *src/i18n/locales/*.

Przełączanie wersji językowej realizowane jest w sposób dynamiczny i asynchroniczny, eliminując konieczność ponownego przeładowania struktury dokumentu DOM przez przeglądarkę internetową. Jako dowód poprawnego działania mechanizmu umiędzynarodowienia (ang. *internationalization*), na opisanym wcześniej rysunku 8 zaprezentowano widok panelu przesyłania zadań z aktywnym językiem angielskim.

6.6 Zarządzanie motywami graficznymi interfejsu

Aplikacja kliencka w pełni obsługuje dwa warianty wizualne interfejsu użytkownika: motyw jasny (ang. *light mode*) oraz ciemny (ang. *dark mode*). Architektura tego mechanizmu bazuje na natywnych zmiennych CSS (ang. *CSS custom properties*), które zdefiniowano w selektorach *:root* oraz *.dark*. Są one dynamicznie interpretowane i wstrzykiwane przez silnik frameworka *Tailwind CSS*.

Bieżąca preferencja operatora dotycząca wybranego schematu kolorów jest utrwalana w pamięci podręcznej `localStorage` z wykorzystaniem mechanizmu automatycznej persistencji stanu globalnego biblioteki *Zustand*. Pozwala to na zachowanie wybranego wariantu przy kolejnych sesjach użytkownika. Dla porównania walorów wizualnych, na przytoczonym wcześniej rysunku 10 przedstawiono interfejs panelu administracyjnego z aktywnym motywem jasnym, podczas gdy pozostałe ekrany zobrazowano w domyślnym wariacie ciemnym.

7 Agent sprzętowy — AddiPi Agent

7.1 Opis ogólny

Komponent *AddiPi Agent* stanowi lokalną warstwę oprogramowania uruchamianą bezpośrednio na mikrokomputerze Raspberry Pi. Pełni on rolę dwukierunkowego pośrednika (ang. *proxy*) pomiędzy rozproszoną architekturą chmurową (usługami *Azure IoT Hub* oraz *Azure Blob Storage*) a niskopoziomowym, lokalnym interfejsem sterującym drukarki 3D, którym zarządza serwer *OctoPrint*.

Agent został zaimplementowany w języku Python. Architektura operacyjna tego komponentu została zaprojektowana w taki sposób, aby funkcjonował on jako proces działający w pętli ciągłej. W zależności od konfiguracji docelowej, jego cyklem życia zarządza demon systemowy oprogramowania układowego `systemd` lub odizolowany kontener środowiska *Docker*.

7.2 Struktura obiektowa i architektura klas

W celu zapewnienia wysokiej modularności oraz łatwości rozbudowy kodu źródłowego, architekturę agenta podzielono na trzy główne klasy o ściśle zdefiniowanych odpowiedzialnościach (zgodnie z zasadą pojedynczej odpowiedzialności):

- **PrinterAgent** — implementuje nadrzędną logikę biznesową aplikacji brzegowej. Odpowiada za utrzymywanie stałego połączenia z usługą *Azure IoT Hub*, asynchroniczny pobór plików sterujących z repozytorium chmurowego, transmisję strumienia danych telemetrycznych oraz nadrzędne monitorowanie stanów procesu produkcyjnego.
- **OctoPrintClient** — stanowi warstwę abstrakcji nad interfejsem sieciowym REST API systemu *OctoPrint*. Klasa ta enkapsuluje operacje wejścia-wyjścia odpowiedzialne za odczyt bieżących temperatur i stanów mechanicznych drukarki, buforowanie plików w pamięci lokalnej urządzenia, a także bezpośrednie sterowanie procesem wytwórczym (rozpoczynanie, wstrzymywanie i anulowanie druku).
- **ConfigLoader** — komponent pomocniczy odpowiedzialny za bezpieczne wczytywanie parametrów startowych, sekretów chmurowych oraz zmiennych środowiskowych utrwalonych w pliku konfiguracyjnym `.env`.

7.3 Połączenie z IoT Hub

Agent sprzętowy inicjuje i utrzymuje stałe połączenie z usługą *Azure IoT Hub* za pośrednictwem protokołu MQTT, wykorzystując do tego celu unikalny ciąg uwierzytelniający urządzenia (ang. *device connection string*). Na listingu 12 przedstawiono fragment kodu odpowiedzialny za instancjonowanie asynchronicznego klienta oraz rejestrację funkcji zwrotnej (ang. *callback function*) dedykowanej do obsługi przychodzących żądań RPC.

```
1     self.iot_client = IoTHubDeviceClient.  
    create_from_connection_string(  
2         device_connection_string  
3     )  
4  
5     self.iot_client.connect()  
6  
7     self.iot_client.on_method_request_received = (  
8         self.handle_method_request  
9     )
```

Listing 12: Inicjalizacja klienta usługi Azure IoT Hub (*printer_agent.py*)

7.4 Obsługiwane metody bezpośrednie

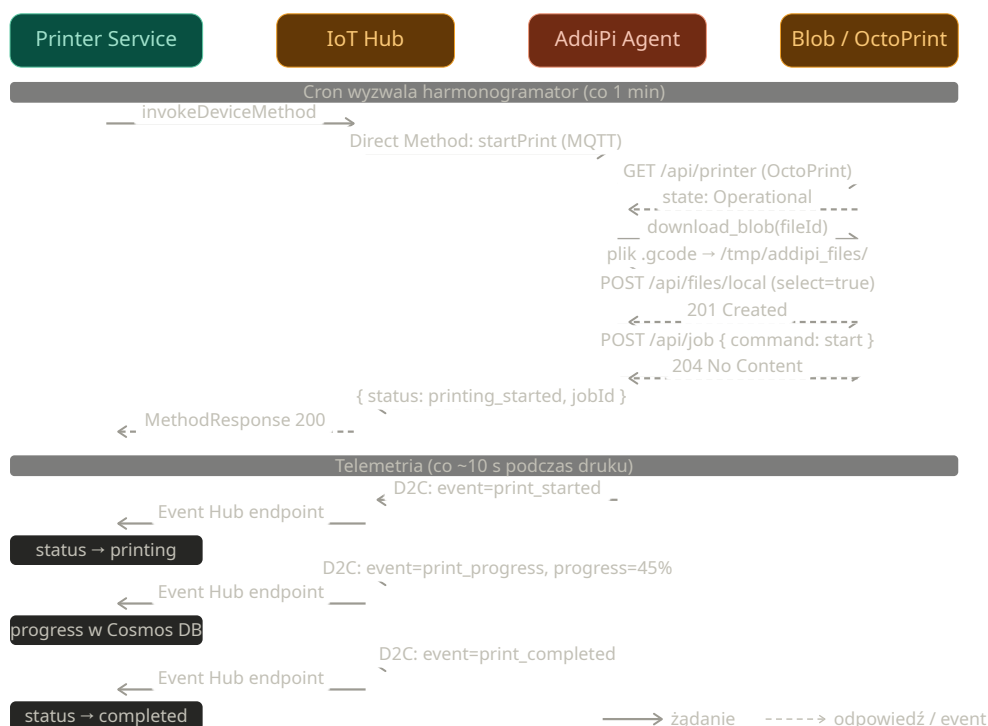
Komponent realizuje zestaw metod bezpośrednich (ang. *Direct Methods*) udostępnianych przez chmurę obliczeniową. W tabeli 9 wyszczególniono metody zaimplementowane w strukturze agenta wraz z ich parametrami wejściowymi oraz opisem wywoływanych akcji.

Tabela 9: Metody bezpośrednie obsługiwane przez agenta sprzętowego.

Metoda	Parametry wejściowe	Opis operacji systemowej
startPrint	fileId, jobId	Pobranie pliku z Azure Blob Storage, przesłanie go do <i>OctoPrint</i> oraz uruchomienie procesu drukowania
cancelPrint	—	Anulowanie aktywnego procesu wytwórczego za pośrednictwem interfejsu API systemu <i>OctoPrint</i>
getStatus	—	Pobranie aktualnego stanu operacyjnego drukarki, postępu procentowego zadania oraz odczytów termicznych urządzenia

7.5 Przebieg obsługi metody bezpośredniej startPrint

Inicjalizacja procesu wytwórczego przez system chmurowy opiera się na synchronicznym wywołaniu metody bezpośredniej `startPrint`. Na rysunku 11 zaprezentowano diagram sekwencji obrazujący przepływ sterowania oraz interakcje pomiędzy agentem sprzętowym, usługami chmurowymi a serwerem *OctoPrint*.



Rysunek 11: Diagram sekwencji obsługi metody bezpośredniej `startPrint` przez *Azure IoT Hub*.

Szczegółowy proces obsługi omawianego żądania realizowany jest sekwencyjnie według następującego scenariusza:

1. Agent odbiera żądanie *Direct Method* zawierające parametry `fileId` oraz `jobId`.
2. Następuje sprawdzenie stanu operacyjnego oraz gotowości mechanicznej drukarki 3D przy użyciu metody `OctoPrintClient.is_printer_ready()`.
3. Plik G-code o wskazanym identyfikatorze jest pobierany z *Azure Blob Storage* i buforowany w lokalnym katalogu tymczasowym `/tmp/addipi_files/`.
4. Agent przesyła pobrany plik do systemu *OctoPrint* poprzez wywołanie punktu koń-

cowego POST `/api/files/local` z parametrem `select=true`. (co wymusza automatyczny wybór pliku do druku).)

5. Wyzwolenie fizycznego procesu drukowania realizowane jest poprzez zapytanie do punktu końcowego POST `/api/job` z poleceniem strukturalnym `start`.
6. Agent wysyła zdarzenie telemetryczne `print_started` do usługi *Azure IoT Hub*.

7.6 Transmisja danych telemetrycznych

W trakcie aktywnego procesu produkcyjnego, agent sprzętowy w sposób ciągły monitoruje stan urządzenia w ramach dedykowanej pętli sprawdzającej. Dane telemetryczne są transmitowane do usługi *Azure IoT Hub* cyklicznie z interwałem 10 sekund. Struktura oraz format przesyłanego komunikatu w formacie JSON zostały szczegółowo zaprezentowane na listingu 13.

```
1  {
2    "event": "print_progress",
3    "timestamp": "2025-12-01T14:30:00.000000",
4    "deviceId": "raspberrypi-mkt-01",
5    "jobId": "1764722864093",
6    "progress": 45.5,
7    "printTime": 1800,
8    "printTimeLeft": 2200,
9    "state": "Printing"
10 }
```

Listing 13: Format struktury wiadomości telemetrycznej transmitowanej do chmury.

7.7 Niskopoziomowa komunikacja z serwerem OctoPrint

Klasa `OctoPrintClient` hermetyzuje i realizuje komunikację sieciową z systemem *OctoPrint* w ramach lokalnej sieci komputerowej (z wykorzystaniem lokalnego mapowania adresów, np. `http://prusai3mk3.local`). Autoryzacja i bezpieczeństwo zapytań HTTP są zapewniane poprzez wstrzykiwanie unikalnego klucza dostępowego do nagłówka `X-API-Key` każdego żądania.

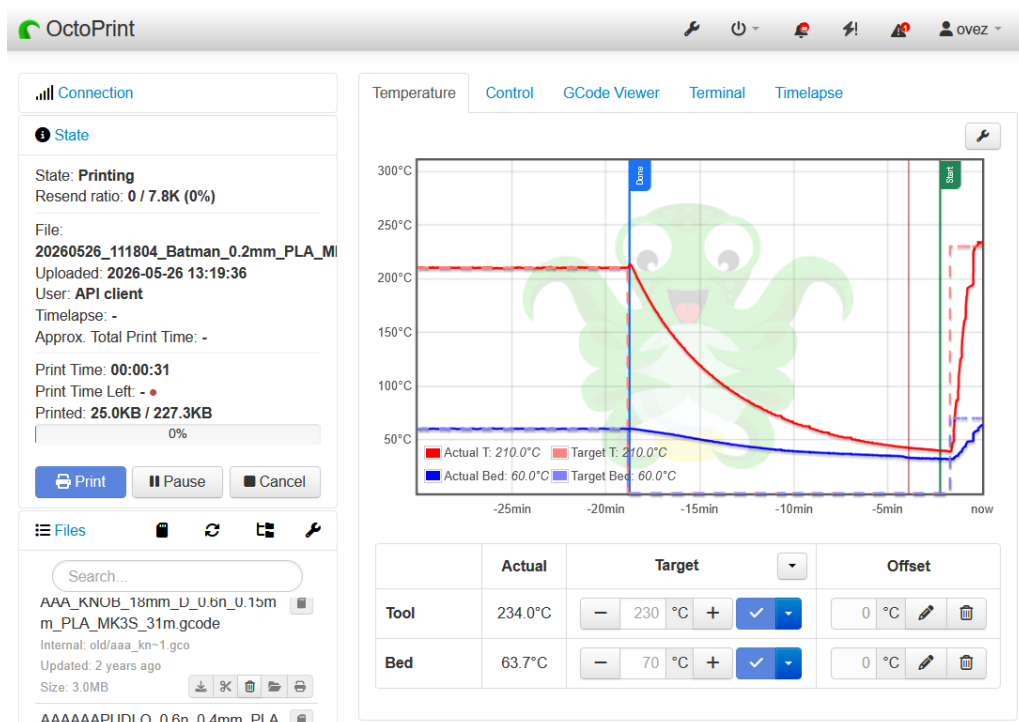
Na listingu 14 zestawiono kluczowe operacje zaimplementowane w strukturze omawianej klasy wraz z przypisanymi do nich punktami końcowymi (ang. *endpoints*) oraz

strukturami poleceń lokalnego interfejsu REST API.

```
1 // Definicja kluczowych metod wchodzących w skład klasy
   OctoPrintClient:
2
3 get_printer_state()      ---> GET /api/printer
4 upload_and_select_file() ---> POST /api/files/local
5 start_print()            ---> POST /api/job { "command": "
   start" }
6 cancel_print()           ---> POST /api/job { "command": "
   cancel" }
7 get_job_info()           ---> GET /api/job
```

Listing 14: Struktura metod klasy OctoPrintClient i ich mapowanie na punkty końcowe API.

W celu zobrazowania środowiska niskopoziomowego, z którym bezpośrednio integruje się opisywany agent sprzętowy, na rysunku 12 przedstawiono natywny panel graficzny oprogramowania serwerowego *OctoPrint*. Widok ten odzwierciedla stan aktywnego procesu drukowania, z którego klasa OctoPrintClient asynchronicznie parsuje metryki czasu, postępu oraz bieżące parametry termiczne dyszy i stołu roboczego.



Rysunek 12: Interfejs systemu *OctoPrint* z widokiem aktywnego procesu druku.

8 Projekt mechaniczny obudowy i pozycjonowania zestawu

8.1 Cel i zakres projektu mechanicznego

Kompletna implementacja systemu *AddiPi* — poza warstwą stricte programową — wymaga fizycznego osadzenia mikrokomputera Raspberry Pi w bezpośrednim sąsiedztwie urządzenia wytwórczego wraz z integracją modułu kamery cyfrowej monitorującej stół roboczy drukarki 3D.

W ramach niniejszej pracy inżynierskiej opracowano, a następnie wytworzono technologią przyrostową dedykowany zestaw komponentów mechanicznych. Został on podzielony na dwa główne moduły strukturalne:

- **CASE** — obudowa ochronna mikrokomputera Raspberry Pi zintegrowana z gniazdem montażowym modułu kamery brzegowej,
- **STAND** — regulowane ramię pozycjonujące (uchwyt), umożliwiające swobodną zmianę kąta widzenia i stabilizację zestawu względem pola roboczego drukarki 3D.

Trójwymiarowe modele geometryczne (CAD) opracowano w programie *SolidWorks* (format .SLDPRT dla pojedynczych części oraz .SLDASM dla złożeń).

W celu zapewnienia otwartości projektu, w repozytorium zasobów systemu — obok natywnych formatów oprogramowania *SolidWorks* — umieszczono modele w neutralnym formacie wymiany danych .STEP, umożliwiającym dalszą edycję w innych środowiskach CAD, a także pliki .3MF i .gcode przygotowane do wydruku na drukarce Prusa MK3S wyposażonej w dyszę o średnicy 0,6 mm z wykorzystaniem filamentu PLA.

8.2 Moduł CASE — obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

Moduł CASE nie został zaprojektowany od podstaw — jako bazę konstrukcyjną wykorzystano otwarty projekt udostępniony w serwisie *GrabCAD* przez *219 Design* [19], opublikowany na licencji **CC BY 4.0**.

Oryginalny model geometryczny dedykowany był dla platformy **Raspberry Pi 4 Model B**. Ze względu na zastosowanie w systemie *AddiPi* urządzenia **Raspberry Pi 3 Model B**, konieczne było wprowadzenie szeregu modyfikacji konstrukcyjnych.

8.2.2 Zakres modyfikacji geometrycznych

Głównym zadaniem w ramach adaptacji modelu było przeprojektowanie komponentu BottomShell, stanowiąca część korpusu obudowy.

Najistotniejsze różnice konstrukcyjne, które determinowały konieczność modyfikacji makra bryłowego obudowy, obejmowały następujące aspekty:

- **Interfejs zasilania** — zmianę standardu i lokalizacji złącza zasilającego z formatu USB-C (stosowanego w nowszej architekturze generacji czwartej) na gniazdo micro-USB, charakterystyczne dla wersji Raspberry Pi 3,
- **Układ portów USB-A i LAN** — odwrócenie i przesunięcie pozycji bloków złączy USB oraz gniazda ethernetowego RJ-45 względem krawędzi płytki PCB,
- **Złącza peryferyjne i multimedialne** — całkowitą zmianę położenia oraz gabarytów pełnowymiarowego portu HDMI i dedykowanego złącza taśmy interfejsu kamery (CSI).

Projekt geometryczny uwzględniał fakt, że obie platformy charakteryzują się identycznymi wymiarami zewnętrznymi laminatu bazowego ($85,0 \times 56,0$ mm) oraz zachowują ten sam standard rozstawu otworów montażowych ($58,0 \times 49,0$ mm, dedykowany dla śrub i tulei dystansowych w rozmiarze M2.5). Pozwoliło to na zachowanie oryginalnej struktury geometrycznej górnej pokrywy obudowy oraz elementów mocujących.

Proces modyfikacji modelu BottomShell w środowisku *SolidWorks* polegał na usunięciu dotychczasowych operacji wyciągnięć wycięć i zastąpieniu ich nowymi operacjami szkicu, sparametryzowanymi pod kątem rzeczywistego rozmieszczenia złączy Raspberry Pi 3.

8.2.3 Integracja strukturalna i mocowanie modułu kamery brzegowej

Obudowa CASE została wyposażona w zintegrowany uchwyt modułu kamery Raspberry Pi (w wersji v1 lub v2, wyposażonej odpowiednio w sensor *OV5647* lub *IMX219*).

Uchwyt pozycjonuje kamerę pod stałym kątem skierowanym w stronę pola roboczego drukarki 3D. Taśma sygnałowa interfejsu CSI prowadzona jest wewnątrz obudowy za pomocą dedykowanego kanału kablowego.

8.2.4 Struktura plików i organizacja zasobów modułu CASE

W celu zapewnienia przejrzystości oraz ułatwienia ewentualnej replikacji lub dalszej modyfikacji projektu mechanicznego, zasoby cyfrowe powiązane z modułem CASE zostały ustrukturyzowane w dedykowanym repozytorium. W tabeli 10 zestawiono strukturę katalogów oraz opisano zawartość poszczególnych zbiorów danych.

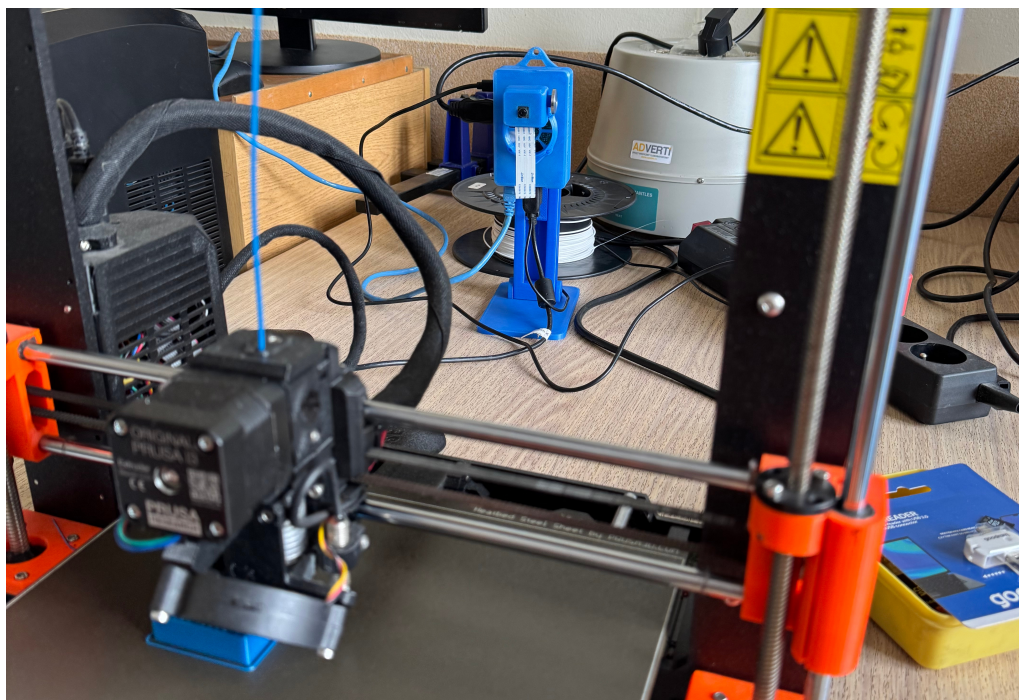
Tabela 10: Struktura katalogów i plików produkcyjnych modułu CASE

Katalog / plik	Zawartość
CASE/files/	Edytowalne modele SolidWorks (.SLDPRT, .SLDASM) oraz modele w formacie wymiany .STEP
CASE/gcode/	Wygenerowane pliki G-code przygotowane dla drukarki Prusa MK3S z dyszą 0,6 mm i filamentem PLA
CASE/Ready To Print/	Modele .3MF oraz .STL przygotowane do otwarcia w programie PrusaSlicer
CASE/grabcad/	Oryginalne pliki referencyjne projektu GrabCAD dla Raspberry Pi 4

8.3 Moduł STAND — regulowany mechanizm pozycjonujący

Moduł STAND stanowi zaprojektowany od podstaw, autorski mechanizm wsporczy, umożliwiający precyzyjne pozycjonowanie obudowy CASE wraz z kamerą względem ramy kinematycznej drukarki 3D.

Konstrukcja ramienia opiera się na połączeniach przegubowych i zapewnia dwa stopnie swobody. Układ ten pozwala na płynną regulację kąta nachylenia osi optycznej obiektywu w płaszczyźnie pionowej oraz dostosowanie wysokości punktu akwizycji obrazu. Dzięki temu możliwe jest optymalne dopasowanie pola widzenia (ang. *Field of View*, FOV) do specyfiki geometrii przestrzeni roboczej danego urządzenia. Fizyczne wdrożenie gotowego zestawu montażowego zamontowanego przy drukarce 3D przedstawiono na rysunku 13.



Rysunek 13: Gotowa konstrukcja mechaniczna zainstalowana przy urządzeniu wytwórczym.

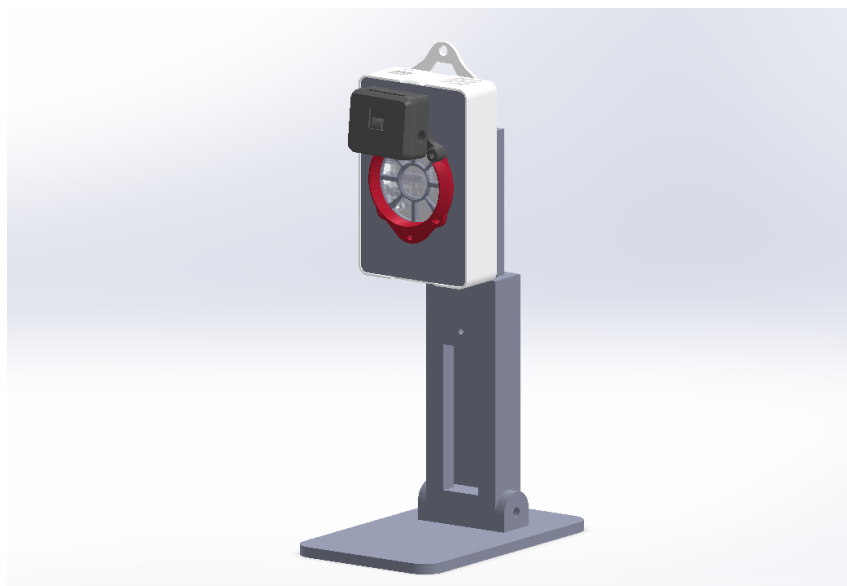
Parametryczne modele źródłowe środowiska *SolidWorks* są przechowywane w dedykowanym katalogu `STAND/own parts/`. Z kolei pliki przygotowane do bezpośredniego procesu przygotowania druku (w formatach `.3MF` oraz `.STL`) zlokalizowano w folderze `STAND/Ready To Print/`, a skompilowane instrukcje sterujące G-code umieszczono w katalogu `STAND/gcode/`.

8.4 Złożenie kompletnego modelu geometrycznego

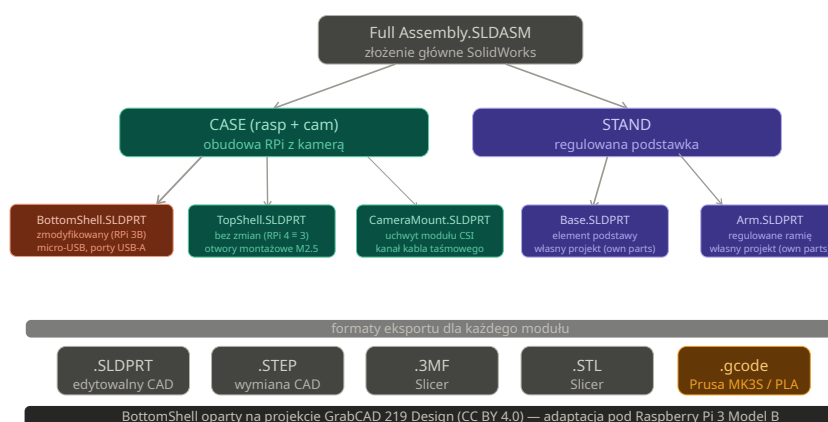
Plik źródłowy `CAD/Full Assembly.SLDASM` zawiera kompletne złożenie (ang. *assembly*) obu zaprojektowanych modułów (`CASE` oraz `STAND`) wraz z zaimportowanym modelem mikrokomputera Raspberry Pi, pełniącym funkcję komponentu referencyjnego. Definicja relacji montażowych (ang. *mates*) wewnątrz struktury modelu umożliwia weryfikację poprawności dopasowania tolerancji wymiarowych, przeprowadzenie analizy potencjalnych kolizji geometrycznych oraz kontrolę parametrów montażowych przed rozpoczęciem docelowego procesu wytwarzania przyrostowego.

Wizualizację trójwymiarową tak przygotowanego środowiska projektowego przedstawiono na rysunku 14. Z kolei na rysunku 15 zaprezentowano strukturalny schemat hierarchii plików i formatów eksportu, który szczegółowo obrazuje architekturę całego pakietu

mechanicznego — od parametrycznych plików źródłowych części, przez fazę przygotowania wydruku (slicing), aż po końcowy kod maszynowy.



Rysunek 14: Trójwymiarowy widok złożeniowy modułów CASE oraz STAND w środowisku CAD.



Rysunek 15: Struktura hierarchiczna złożenia, komponentów oraz formatów produkcyjnych modułów CAD.

8.5 Parametry wydruku

W celu zapewnienia powtarzalności geometrycznej, odpowiedniej wytrzymałości mechanicznej struktury oraz optymalizacji czasu produkcji elementów obudowy, zdefiniowano zestaw rekomendowanych parametrów procesu technologicznego. W tabeli 11 zestawiono szczegółowe nastawy CAM przygotowane dla środowiska programu typu slicer.

Tabela 11: Zalecane parametry druku 3D elementów obudowy systemu.

Parametr	Wartość
Materiał	PLA
Średnica dyszy	0,6 mm
Wysokość warstwy	0,2 mm (profil <i>Quality</i>)
Wypełnienie	20% (wzór Gyroid)
Podpory	Niewymagane dla większości modeli
Orientacja wydruku	Zgodna z konfiguracją w plikach .3MF
Drukarka referencyjna	Prusa MK3S

8.6 Procedura montażu zestawu mechanicznego

Proces montażu oraz kalibracji przestrzennej zestawu komponentów mechanicznych realizowany jest sekwencyjnie według następującego scenariusza:

1. **Wytworzenie komponentów:** Wydrukowanie elementów składowych obudowy i ramienia na podstawie plików sterujących .gcode lub projektów technologicznych .3MF oraz .STL zlokalizowanych w katalogach Ready To Print.
2. **Instalacja elektroniki:** Zamontowanie mikrokomputera Raspberry Pi w dolnej części obudowy (BottomShell) z wykorzystaniem czterech śrub montażowych M2.5 oraz dedykowanych nakrętek.
3. **Integracja magistrali CSI:** Podłączenie taśmy sygnałowej interfejsu CSI do modułu kamery oraz precyzyjne przeprowadzenie jej przez dedykowany przepust osłony w korpusie obudowy.
4. **Aseblacja korpusu:** Zamknięcie obudowy poprzez nałożenie górnej pokrywy (TopShell) oraz mechaniczne zabezpieczenie konstrukcji za pomocą zintegrowanych zatrzasków geometrycznych.
5. **Połączenie modułów:** Osadzenie zmontowanego modułu CASE na regulowanym ramieniu wsporczym STAND wraz z nastawieniem wymaganego kąta pochylenia osi optycznej.
6. **Kalibracja pola widzenia:** Ustawienie kompletnego zestawu w sąsiedztwie drukarki 3D w sposób zapewniający bezkolizyjną pracę maszyn oraz pełną akwizycję geometryczną całego pola roboczego urządzenia.

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

Mikrokomputer *Raspberry Pi 3 Model B* pełni funkcję węzła brzegowego systemu *AddiPi*. Odpowiada on za trzy kluczowe zadania:

- uruchamianie serwera *OctoPrint* komunikującego się z drukarką przez interfejs USB,
- wykonywanie kodu agenta *AddiPi Agent* zaimplementowanego w języku Python,
- udostępnianie strumienia wideo z kamery CSI do zewnętrznej usługi *Video Service*.

Bazy systemowej dostarcza *OctoPi* — oficjalny, minimalistyczny obraz *Raspberry Pi OS Lite* z preinstalowanym środowiskiem *OctoPrint*. Interpreter *Python 3* oraz menedżer *pip* są w tej dystrybucji dostępne domyślnie, co wyeliminowało potrzebę konfiguracji środowiska bazowego.

9.2 Instalacja agenta

Procedura instalacji komponentu *AddiPi Agent* sprowadza się do pobrania zasobów z repozytorium systemowego, wdrożenia pakietów zależnych oraz konfiguracji zmiennych środowiskowych. Sekwencję wymaganych poleceń powłoki systemu operacyjnego zaprezentowano na listingu 15.

```
1  # Klonowanie repozytorium projektu
2  git clone https://github.com/AddiPii/AddiPi-Agent.git
3  cd AddiPi-Agent
4
5  # Instalacja zależności Python
6  python3 -m pip install -r requirements.txt
7
8  # Konfiguracja zmiennych środowiskowych
9  cp .env.example .env
10 nano .env
11
12 # Uzupełnienie wymaganych parametrów:
13 # DEVICE_CONNECTION_STRING, STORAGE_CONN
```

```
14 # OCTOPRINT_API_KEY, OCTOPRINT_URL
```

Listing 15: Instalacja i inicjalizacja agenta na platformie Raspberry Pi.

9.3 Automatyzacja uruchamiania za pomocą systemd

W celu zapewnienia bezobsługowości systemu brzegowego i automatycznego restartu aplikacji po awarii lub restarcie mikrokomputera, skonfigurowano dedykowaną usługę *systemd*. Plik konfiguracyjny jednostki (ang. *unit file*) zlokalizowano w ścieżce `/etc/systemd/system/agent.service`, a jego strukturę zaprezentowano na listingu 16.

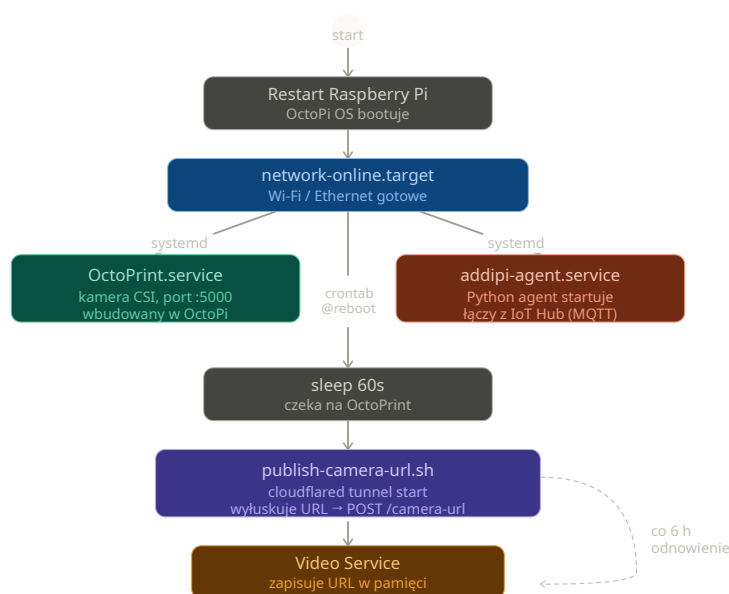
```
1 [Unit]
2 Description=AddiPi Printer Agent
3 After=network-online.target
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=pi
9 WorkingDirectory=/home/pi/AddiPi-Agent
10 ExecStart=/usr/bin/python3 src/app.py
11 Restart=always
12 RestartSec=10
13 EnvironmentFile=/home/pi/AddiPi-Agent/.env
14
15 [Install]
16 WantedBy=multi-user.target
```

Listing 16: Plik jednostki konfiguracji `addipi-agent.service` dla menedżera *systemd*.

Dyrektywa `After=network-online.target` gwarantuje uruchomienie skryptu dopiero po poprawnym zainicjowaniu stosu sieciowego. Zapobiega to błędom gniazd sieciowych (ang. *sockets*) podczas próby nawiązania połączenia przez protokół MQTT z usługą *Azure IoT Hub*. Aktywację i uruchomienie demona realizuje się sekwencyjnie za pomocą poleceń systemowych: `sudo systemctl daemon-reload`, `sudo systemctl enable addipi-agent` oraz `sudo systemctl start addipi-agent`. Bieżący stan procesu weryfikowany jest instrukcją `sudo systemctl status addipi-agent`.

Kompletną sekwencję zdarzeń oraz zależność potoków rozruchowych usług systemowych i skryptów harmonogramu po restarcie platformy sprzętowej przedstawiono na

rysunku 16.



Rysunek 16: Diagram potoku rozruchowego mikrokomputera po restarcie systemu.

9.4 Udostępnianie strumienia wideo przez Cloudflare Tunnel

Lokalny strumień wideo MJPEG, generowany jest pod adresem:

`http://localhost:8080/?action=stream`

Połączenie to jest domyślnie zablokowane poza strukturą sieci lokalnej. W celu umożliwienia zewnętrznego podglądu z kamery w aplikacji klienckiej (hostowanej na platformie *Vercel*), zastosowano technologię *Cloudflare Tunnel* (*cloudflared*) [20].

Narzędzie *cloudflared* zestawia szyfrowane, bezpieczne połączenie wychodzące pomiędzy mikrokomputerem a infrastrukturą *Cloudflare*. Pozwala to na bezpłatne i stabilne zmapowanie lokalnego portu na publiczny adres URL w domenie *trycloudflare.com* (np. `https://<losowy-hash>.trycloudflare.com`) bez konieczności konfiguracji publicznego adresu IP na urządzeniu brzegowym czy modyfikacji zapór sieciowych.

9.4.1 Instalacja narzędzia cloudflared

Wdrożenie oficjalnego klienta *cloudflared* na platformie sprzętowej z procesorem o architekturze ARM wymaga pobrania skompilowanego pliku binarnego, nadania mu uprawnień wykonywania oraz relokacji do katalogu systemowego. Operacje te realizuje się sekwencyjnie w terminalu systemu operacyjnego za pomocą następujących instrukcji:

```
wget https://github.com
chmod +x cloudflared-linux-arm64
sudo mv cloudflared-linux-arm64 /usr/local/bin/cloudflared
cloudflared -version
```

Ostatnie polecenie wykonuje weryfikację poprawności instalacji poprzez zwrócenie bieżącej wersji oprogramowania w konsoli.

9.4.2 Skrypt automatyzacji publikacji adresu sieciowego

Po uruchomieniu tunelu Cloudflare usługa cloudflared przydziela nowy, losowy publiczny adres URL. Aby serwis Video Service posiadał zawsze aktualny adres strumienia kamery, opracowano skrypt `publish-camera-url.sh`, który realizuje następujące operacje:

1. Uruchomienie tunelu cloudflared tunnel w tle oraz przekierowanie logów do pliku tymczasowego.
2. Odczekanie kilku sekund na zestawienie połączenia tunelowego.
3. Odczytanie przydzielonego adresu URL z pliku logów na podstawie wzorca domeny trycloudflare.com.
4. Przesłanie uzyskanego adresu URL do serwisu Video Service za pomocą żądania POST /camera-url.

Kompletny kod źródłowy tego mechanizmu zaprezentowano na listingu 17.

```
1  #!/bin/bash
2  LOG_FILE="/tmp/cloudflared.log"
3  VIDEO_SERVICE_URL="http://azure.com"
4  CAMERA_LOCAL_URL="http://localhost:8080"
5
6  # Uruchomienie tunelu w tle i przekierowanie strumieni danych
7  cloudflared tunnel --url "$CAMERA_LOCAL_URL" > "$LOG_FILE" 2>&1 &
8
9  # Iteracyjne sondowanie logu w celu ekstrakcji adresu (maks. 30 s
10 )
    TUNNEL_URL=""
```

```

11  for i in $(seq 1 15); do
12      sleep 2
13      TUNNEL_URL=$(grep -E -o 'https://[a-zA-Z0-9-]*\.trycloudflare\.
        com' "$LOG_FILE" | head -1)
14      if [ -n "$TUNNEL_URL" ]; then
15          break
16      fi
17      done
18
19      if [ -z "$TUNNEL_URL" ]; then
20          exit 1
21      fi
22
23      # Transmisja adresu do centralnej usługi Video Service
24      curl -s -X POST "$VIDEO_SERVICE_URL/camera-url" \
25      -H "Content-Type: application/json" \
26      -d "{\"url\": \"$TUNNEL_URL\"}"

```

Listing 17: Skrypt automatycznej rejestracji dynamicznego adresu URL tunelu (publish-camera-url.sh).

9.4.3 Automatyzacja odnawiania adresu w harmonogramie cron

Z uwagi na fakt, że bezpłatne tunele w domenie trycloudflare.com wykazują tendencję do samoczynnego wygasania po okresie bezczynności lub wskutek fluktuacji sieciowych, wdrożono mechanizm ciągłego odnawiania sesji. Do tego celu wykorzystano systemowy harmonogram zadań *cron*. W tablicy konfiguracji użytkownika (*crontab*) zdefiniowano dyrektywę *@reboot*, która inicjuje skrypt *publish-camera-url.sh* bezpośrednio po starcie systemu operacyjnego, oraz regułę czasową uruchamiającą procedurę cyklicznie z interwałem sześciogodzinnych iteracji, co zapewnia pełną ciągłość dystrybucji strumienia wideo.

9.5 Konfiguracja harmonogramu zadań *crontab*

Zarządzanie potokami cyklicznymi oraz automatyzację skryptów na platformie Raspberry Pi powierzono systemowemu demonowi *cron*. Edycja harmonogramu realizowana jest poleceniem *crontab -e* w kontekście uprawnień użytkownika *pi*. Wprowadzone wpisy

konfiguracyjne, implementujące opóźniony start rozruchowy oraz cykliczne odnawianie tunelu, przedstawiono na listingu 18.

```
1  # Publikacja adresu URL kamery po restarcie systemu (opóźnienie
    60 s)
2  @reboot sleep 60 && /home/pi/scripts/publish-camera-url.sh >> /
    home/pi/logs/camera-url.log 2>&1
3
4  # Odnawianie adresu URL kamery cyklicznie co 6 godzin
5  0 */6 * * * /home/pi/scripts/publish-camera-url.sh >> /home/pi/
    logs/camera-url.log 2>&1
```

Listing 18: Konfiguracja zadań harmonogramu cron dla użytkownika pi.

9.5.1 Uzasadnienie zastosowanych parametrów

- `@reboot sleep 60` — opóźnienie wynoszące 60 sekund po uruchomieniu systemu jest wymagane, ponieważ środowisko *OctoPi* podczas startu inicjalizuje wiele usług systemowych, m.in. *OctoPrint*, obsługę kamery oraz interfejs sieciowy Wi-Fi. Próba uruchomienia tunelu na wcześniejszym etapie może zakończyć się błędem z powodu niedostępności lokalnego portu strumienia wideo.
- `0 */6 * * *` — cykliczne uruchamianie skryptu co 6 godzin umożliwia weryfikację dostępności tunelu *Cloudflare* oraz ponowną publikację adresu URL w przypadku zerwania połączenia. Wybrany interwał stanowi kompromis pomiędzy częstotliwością odświeżania a ograniczeniem zbędnego obciążenia sieci i usług zewnętrznych.

9.6 Problem dostępności sieci podczas uruchamiania systemu

W trakcie testów zaobserwowano, że zadania uruchamiane przez mechanizm `@reboot` w *crontab* mogą zostać wykonane przed pełną inicjalizacją interfejsu sieciowego. W rezultacie skrypt publikujący adres URL kamery mógł uruchomić się przed uzyskaniem dostępu do sieci lub przed pełnym uruchomieniem usługi *OctoPrint*.

W przypadku agenta *AddiPi* problem rozwiązano z wykorzystaniem dyrektywy `After=network-online.target` w konfiguracji usługi *systemd*. Dla skryptu uruchamianego

przez cron zastosowano dodatkowe opóźnienie (`sleep 60`), stanowiące proste i niezawodne obejście problemu kolejności startu usług.

Alternatywnym rozwiązaniem byłoby zarejestrowanie skryptu publikującego adres URL jako osobnej usługi `systemd`, również zależnej od `network-online.target`. Podejście to zapewniałoby większą kontrolę nad kolejnością uruchamiania komponentów systemu. Przykładową konfigurację jednostki typu `oneshot` zaprezentowano na listingu 19.

```
1  [Unit]
2  Description=AddiPi Camera URL Publisher
3  After=network-online.target addipi-agent.service
4  Wants=network-online.target
5
6  [Service]
7  Type=oneshot
8  User=pi
9  ExecStart=/home/pi/scripts/publish-camera-url.sh
10 RemainAfterExit=no
11
12 [Install]
13 WantedBy=multi-user.target
```

Listing 19: Alternatywna konfiguracja skryptu publikacji jako usługi `systemd`.

9.7 Podsumowanie konfiguracji węzła brzegowego

W tabeli 12 zestawiono oraz sklasyfikowano wszystkie kluczowe usługi systemowe i zadania harmonogramu skonfigurowane w obrębie mikrokomputera Raspberry Pi.

Tabela 12: Podsumowanie usług i zadań cyklicznych na platformie Raspberry Pi.

Usługa / komponent	Mechanizm mieniowy	uruchomienia	Opis realizowanej odpowiedzialności
<i>OctoPrint</i>	<code>systemd</code> (natywny w <i>OctoPi</i>)	w	Niskopoziomowe sterowanie drukarką przez interfejs USB, generowanie lokalnego strumienia wizyjnego.

Tabela 12 – ciąg dalszy z poprzedniej strony

Usługa / komponent	Mechanizm mieniowy	urucho-	Opis realizowanej odpowiedzialności
<i>AddiPi Agent</i>	systemd	(addipi-agent.service)	Utrzymywanie sesji z <i>IoT Hub</i> , obsługa żądań <i>Direct Methods</i> , transmisja danych telemetrycznych.
<i>Cloudflare Tunnel</i>	cron (@reboot)		Zestawienie szyfrowanego połączenia wychodzącego, inicjowane z opóźnieniem 60 s od startu systemu.
Odnowienie adresu URL	cron (0 */6 * * *)		Iteracyjna renegotiacja sesji tunelu oraz przesyłanie aktualnego rejestru do usługi <i>Video Service</i> .

10 Wdrożenie i infrastruktura produkcyjna

10.1 Hybrydowa strategia architektoniczna

Infrastruktura produkcyjna systemu *AddiPi* bazuje na paradygmacie wielochmurowym (ang. *Multi-Cloud*). Podział odpowiedzialności pomiędzy dostawców zasobów został podyktowany ich specyfikacją techniczną, modelami usługowymi oraz dostępnością bezpłatnych pakietów deweloperskich (ang. *free tier*).

W tabeli 13 zestawiono szczegółową strukturę lokowania poszczególnych komponentów systemu, przypisane im platformy sprzętowo-programowe oraz modele dostarczania usług.

Tabela 13: Podział infrastruktury produkcyjnej systemu *AddiPi*.

Platforma / dostawca	Model usługowy	Hostowane komponenty ekosystemu
<i>Microsoft Azure</i>	PaaS / usługi zarządzane	Warstwa danych i integracji: <i>Cosmos DB</i> , <i>Blob Storage</i> , <i>Service Bus</i> oraz <i>IoT Hub</i> .
<i>Oracle Cloud (OCI)</i>	IaaS (Maszyna wirtualna)	Kontenery <i>Docker</i> <i>Compose</i> : <i>Auth</i> , <i>User</i> , <i>Files</i> , <i>Queue</i> oraz <i>Printer Service</i> .
<i>Vercel</i>	Serverless / Jamstack	Aplikacja kliencka (React) oraz bezserwerowa usługa dystrybucji wideo (<i>Video Service</i>).
<i>Raspberry Pi</i>	On-premise (Brzegowe)	Lokalny sterownik <i>OctoPrint</i> oraz demon <i>AddiPi Agent</i> .

Przyjęta topologia wynika bezpośrednio z limitów platformy *Vercel*, która uniemożliwia egzekucję procesów długotrwale działających (ang. *long-running processes*), takich jak stałe nasłuchiwanie kolejki *Service Bus* czy cykliczne zadania orkiestracji. Komponenty wymagające ciągłej aktywności ulokowano na bezpłatnej instancji VM. Standard. E2.1.Micro w chmurze *Oracle* [17], podczas gdy *Microsoft Azure* dostarcza standaryzowane, w pełni zarządzane interfejsy zapisu oraz wymiany komunikatów.

10.2 Zarządzane usługi integracji danych i komunikacji chmurowej

Wszystkie zasoby dostarczane przez platformę *Microsoft Azure* funkcjonują wyłącznie w modelu PaaS, stanowiąc scentralizowany węzeł wymiany informacji i persystencji danych

dla komponentów uruchomionych w infrastrukturze *Oracle Cloud* oraz na mikrokomputerze brzegowym. W tabeli 14 wyszczególniono parametry wdrożeniowe oraz specyfikację konfiguracyjną poszczególnych usług chmurowych.

Tabela 14: Specyfikacja konfiguracji usług Microsoft Azure wykorzystanych w projekcie.

Nazwa usługi	Rola architektoniczna	Parametry konfiguracyjne i plany
<i>Azure Cosmos DB</i>	Baza danych NoSQL	Interfejs API SQL, baza addipi, kontenery: users, jobs, refresh-tokens; klucz partycjonowania: /id.
<i>Azure Blob Storage</i>	Repozytorium obiektowe	Instancja addipifiles, kontener gcode, warstwa wydajnościowa Standard LRS.
<i>Azure Service Bus</i>	Broker komunikatów	Przestrzeń nazw addipisb, kolejka print-queue, warstwa Basic, parametr TTL wiadomości: 14 dni.
<i>Azure IoT Hub</i>	Zarządzanie węzłami IoT	Hub addipi-iothub, tożsamość raspberry-pi-mkt-01, plan bezpłatny F1, protokół MQTT.

10.3 Automatyzacja orkiestracji chmurowej — Terraform

Kompletna infrastruktura chmurowa systemu została zdefiniowana w sposób deklaratywny zgodnie z paradygmatem *Infrastructure as Code* (IaC) przy użyciu narzędzia *Terraform* [16]. Podejście to gwarantuje pełną powtarzalność środowisk i umożliwia odtworzenie zasobów na dowolnych kontach dostawców chmurowych. Repozytorium *AddiPi-Infrastructure* zostało podzielone na dwa autonomiczne moduły:

- terraform/ — konfiguracja zasobów *Microsoft Azure* (grupa zasobów *AddiPi-RG*, *Storage Account*, *Service Bus*, *Cosmos DB* oraz *IoT Hub*) z wykorzystaniem providera hashicorp/azurerm w wersji ~> 4.0,
- terraform-oci/ — konfiguracja zasobów *Oracle Cloud* (sieć vcn-addipi, reguły bezpieczeństwa dla portów: 3001, 3002, 3050, 3070, 5000, instancja VM. Standard.E2.1.Micro o parametrach 1 OCPU i 1 GB RAM oraz publiczny adres IP) z użyciem providera oracle/oci w wersji ~> 6.0.

Oba moduły wykorzystują pliki `imports.tf` zawierające deklaratywne bloki `import`, co pozwala na asymilację istniejącej infrastruktury do pliku stanu (ang. *state file*) bez konieczności niszczenia i ponownego tworzenia obiektów chmurowych.

Proces inicjalizacji dla wybranego środowiska (katalog `terraform/` dla Azure lub `terraform-oci/` dla OCI) polega na przygotowaniu parametrów wejściowych w pliku `terraform.tfvars` (na podstawie dostarczonego szablonu `.example`) i wykonaniu sekwencji instrukcji zaprezentowanych na listingu 20.

```
1  # Inicjalizacja providerów i wdrożenie konfiguracji
2  terraform init
3  terraform plan
4  terraform apply
```

Listing 20: Inicjalizacja i aplikacja planu infrastruktury przez Terraform.

Po pomyślnym zastosowaniu planu, wygenerowane wartości niejawne (ang. *sensitive outputs*), takie jak parametry połączeniowe czy klucze główne baz danych, pobiera się bezpośrednio z poziomu konsoli za pomocą instrukcji typu `terraform output -raw <nazwa_zmiennej>` (np. dla zmiennych `storage_connection_string`, `service_bus_connection_string`, `cosmos_endpoint` oraz `cosmos_key`). Pozyskane w ten sposób sekrety są następnie wstrzykiwane do pliku konfiguracyjnego `.env` na produkcyjnej maszynie wirtualnej OCI.

10.4 Specyfikacja konteneryzacji i optymalizacja obrazów

W celu zapewnienia powtarzalności środowisk uruchomieniowych, każdy z mikroserwisów został odizolowany w strukturze kontenera *Docker*. Dla usług opartych na technologii *Node.js* wdrożono mechanizm wieloetapowego budowania (ang. *multi-stage build*), którego celem jest separacja zależności deweloperskich i procesu kompilacji *TypeScript* (etap *builder*) od lekkiego obrazu produkcyjnego. Wzorcową konfigurację tego potoku dla usługi *Auth Service* zaprezentowano na listingu 21.

```
1  FROM node:20-alpine AS builder
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm ci
5  COPY tsconfig*.json ./
6  COPY src ./src
```

```

7  RUN npm run build
8
9  FROM node:20-alpine
10 WORKDIR /app
11 ENV NODE_ENV=production
12 COPY package*.json ./
13 RUN npm ci --omit=dev
14 COPY --from=builder /app/dist ./dist
15 EXPOSE 3001
16 CMD ["node", "dist/index.js"]

```

Listing 21: Wieloetapowy plik konfiguracyjny Dockerfile serwisu Node.js.

Z kolei usługa *Files Service* (Python) z uwagi na interpretowany charakter kodu wykorzystuje uproszczony, jednoetapowy schemat oparty na dystrybucji minimalistycznej. Strukturę tego pliku, uwzględniającą optymalizację rozmiaru warstw poprzez blokowanie pamięci podręcznej pip, przedstawiono na listingu 22.

```

1  FROM python:3.13-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY app.py .
6  COPY config/ ./config/
7  COPY controllers/ ./controllers/
8  COPY routes/ ./routes/
9  COPY middleware/ ./middleware/
10 EXPOSE 5000
11 CMD ["python", "app.py"]

```

Listing 22: Konfiguracja Dockerfile dla usługi Files Service (Python).

10.5 Automatyzacja procesów CI/CD i Continuous Deployment

W celu zautomatyzowania cyklu wydawniczego oprogramowania, każde repozytorium źródłowe zostało zintegrowane z usługą *GitHub Actions*. Zdefiniowane potoki automatyzacji (ang. *workflows*) są wyzwalane po zdarzeniu push do gałęzi main i realizują następujące etapy:

1. **Kompilacja i budowanie:** Kompilacja kodu źródłowego oraz budowanie produk-

cyjnego obrazu *Docker*.

2. **Tagowanie zasobu:** Nadanie unikalnego tagu rejestru w postaci `ghcr.io/AddiPii/<nazwa-serwisu>:latest`.
3. **Publikacja artefaktu:** Automatyczna publikacja gotowego obrazu wewnątrz *GitHub Container Registry* (GHCR).

Na produkcyjnej maszynie wirtualnej OCI osadzono autonomiczny komponent *Watchtower* [21]. Narzędzie to cyklicznie sonduje rejestr GHCR, weryfikuje sygnatury obrazów i po wykryciu nowej kompilacji automatycznie pobiera warstwy oraz restartuje odpowiednie kontenery. Podejście to w pełni realizuje założenia paradygmatu ciągłego wdrażania (ang. *Continuous Deployment*), uniezależniając proces aktualizacji produkcyjnej od manualnej interwencji administratora na serwerze.

10.6 Orkiestracja kontenerowa warstwy serwerowej — Docker Compose

Uruchomienie mikroservisów backendowych na instancji *Oracle Cloud* realizowane jest przy użyciu narzędzia *Docker Compose*. W środowisku produkcyjnym wykorzystano wariant konfiguracyjny `docker-compose.ghcr.yml`, który pobiera skompilowane artefakty bezpośrednio z rejestru kontenerów GHCR. Rozwiązanie to drastycznie minimalizuje obciążenie procesora i pamięci RAM instancji IaaS, eliminując potrzebę lokalnego budowania obrazów. Każdy z komponentów funkcjonuje w odizolowanym środowisku z dedykowaną polityką restartu `unless-stopped`, a komunikacja wewnętrzna odbywa się w ramach wirtualnej sieci `addipi-network`.

Na listingu 23 przedstawiono reprezentatywny fragment produkcyjnego pliku konfiguracyjnego.

```
1  version: '3.9'
2
3  services:
4    auth:
5      image: ghcr.io/AddiPii/AddiPi-Auth-Service:latest
6      env_file: .env
7      ports:
8        - "3001:3001"
9      restart: unless-stopped
```

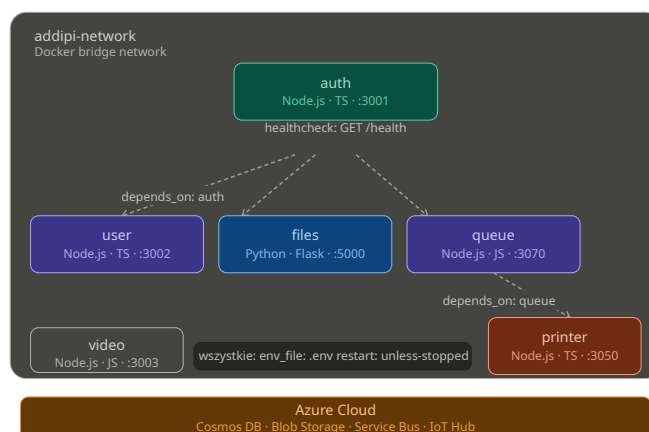
```

10
11     user:
12         image: ghcr.io/AddiPii/AddiPi-User-Service:latest
13         env_file: .env
14         ports:
15         - "3002:3002"
16         depends_on: [auth]
17         restart: unless-stopped
18
19     printer:
20         image: ghcr.io/AddiPii/AddiPi-Printer-Service:latest
21         env_file: .env
22         ports:
23         - "3050:3050"
24         depends_on: [queue, auth]
25         restart: unless-stopped

```

Listing 23: Fragment produkcyjnego pliku konfiguracyjnego docker-compose.ghcr.yml.

Ciągłość operacyjną środowiska zabezpiecza wspomniany wcześniej kontener *Watchtower*, działający w tle jako demon aktualizacyjny. Bezpieczeństwo sieciowe instancji maszyn wirtualnych OCI egzekwowane jest na poziomie warstwy sieciowej chmury, ograniczając ruch przychodzący wyłącznie do niezbędnych portów: 22 (SSH), 3001 (*Auth Service*), 3002 (*User Service*), 3050 (*Printer Service*), 3070 (*Queue Service*) oraz 5000 (*Files Service*). Reguły te są definiowane deklaratywnie w module `terraform-oci/`. Topologię tak zorganizowanej sieci kontenerów oraz ich zależności startowe przedstawiono na rysunku 17.



Rysunek 17: Diagram architektury kontenerów Docker Compose i ich relacji zależności.

10.7 Zarządzanie konfiguracją i sekretami przez zmienne środowiskowe

Konfiguracja serwisów uruchamianych na instancji OCI realizowana jest przy pomocy zmiennych środowiskowych przekazywanych do kontenerów Docker za pośrednictwem pliku `.env`. Mechanizm ten umożliwia oddzielenie konfiguracji od kodu źródłowego, co upraszcza wdrażanie aplikacji w różnych środowiskach oraz zwiększa bezpieczeństwo poprzez eliminację konieczności przechowywania poufnych danych w repozytoriach kodu.

W środowisku produkcyjnym plik `.env` udostępniany jest wszystkim serwisom za pomocą dyrektywy `env_file` w konfiguracji Docker Compose. Repozytorium projektu zawiera również plik `.env.example`, stanowiący szablon wymaganej konfiguracji i dokumentujący znaczenie poszczególnych parametrów.

Tabela 15 przedstawia najważniejsze zmienne środowiskowe wykorzystywane przez system.

Tabela 15: Kluczowe zmienne środowiskowe systemu

Zmienna	Przeznaczenie
COSMOS_ENDPOINT	Adres sieciowy instancji bazy danych.
COSMOS_KEY	Główny klucz autoryzacji i odczytu/zapisu.
STORAGE_CONN	Parametry połączeniowe repozytorium obiektów.
SERVICE_BUS_CONN	Ciąg połączeniowy brokera komunikatów.
IOT_HUB_SERVICE_CS	Interfejs usługowy do wywołań metod RPC.
IOT_HUB_EVENT_HUB_CS	Punkt odbioru asynchronicznej telemetrii.
JWT_SECRET	Klucz podpisywania tokenów dostępu (<i>access</i>).
JWT_REFRESH_SECRET	Klucz podpisywania tokenów odświeżania (<i>refresh</i>).
AUTH_SERVICE_URL	Adres bazowy usługi uwierzytelniania
EMAIL_USER	Konto nadawcze komunikatów aktywacyjnych.
EMAIL_PASSWORD	Hasło aplikacyjne dedykowanej skrzynki e-mail.
DEVICE_CONNECTION_STRING	Token uwierzytelniania urządzenia zarejestrowanego w <i>Azure IoT Hub</i>
OCTOPRINT_API_KEY	Klucz API platformy <i>OctoPrint</i>

10.8 Wdrożenie warstwy prezentacji oraz usługi wideo

Aplikacja kliencka (frontendowa) oraz usługa *Video Service* zostały wdrożone na platformie chmurowej *Vercel* [22], co zapewnia ich globalną dystrybucję za pośrednictwem sieci CDN (ang. *Content Delivery Network*). Proces budowania i aktualizacji zasobów statycznych pod adresem <https://addipi.vercel.app> realizowany jest w pełni automa-

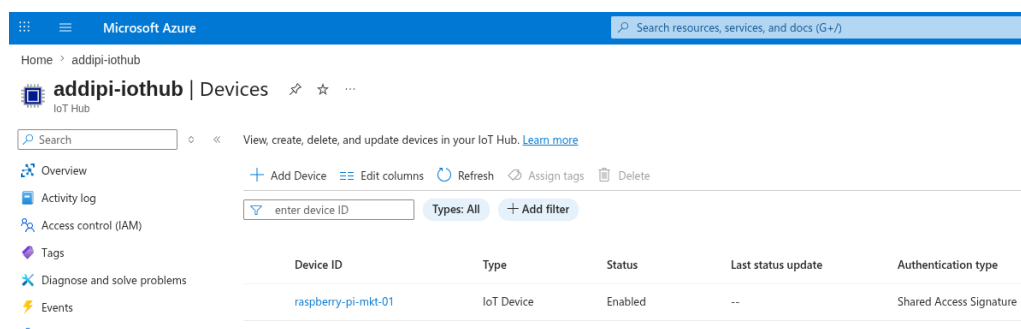
tycznie w modelu CD po każdym zdarzeniu push do gałęzi main repozytorium *GitHub*, uruchamiając komendę produkcyjną `npm run build`.

Specyfika platformy *Vercel* opiera się na paradygmacie bezserwerowym (ang. *Serverless*), który narzuca sztywne ograniczenia czasowe na egzekucję kodu (ang. *execution timeouts*). Z tego względu infrastruktura ta została wykorzystana wyłącznie do hostowania interfejsu użytkownika oraz lekkiej, bezstanowej usługi *Video Service*, przechowującej adres URL strumienia w pamięci RAM. Mikroserwisy wymagające ciągłego, asynchronicznego nasłuchiwania kolejek komunikatów (*Queue Service*) oraz orkiestracji zadań telemetrycznych (*Printer Service*) charakteryzują się naturą procesów długotrwale działających (ang. *long-running processes*). Ich implementacja w modelu *Serverless* byłaby niemożliwa, co w pełni uzasadnia ich lokowanie na dedykowanej maszynie wirtualnej OCI.

10.9 Wdrożenie komponentu brzegowego

Wdrożenie komponentu *AddiPi Agent* zrealizowano bezpośrednio w środowisku lokalnym mikrokomputera Raspberry Pi. Zgodnie ze szczegółowym opisem zawartym w rozdziale 9, proces ten oparto na demonie systemowym `systemd (addipi-agent.service)`. Konfiguracja ta gwarantuje pełną autonomię węzła brzegowego, realizując automatyczny rozruch skryptów po starcie zasilania oraz deterministyczne wznowianie pracy w przypadku awarii procesu. Zapewnia to bezobsługową i niezawodną integrację lokalnego sterownika *OctoPrint* z usługami chmurowymi.

Bezpośrednie, produkcyjne potwierdzenie poprawnej rejestracji oraz aktywnego stanu połączenia urządzenia brzegowego w chmurze obliczeniowej zaprezentowano na rysunku 18, przedstawiającym panel administracyjny usługi *Azure IoT Hub*.



Rysunek 18: Widok zarejestrowanego i połączanego mikrokomputera w panelu usługi Azure IoT Hub.

11 Podsumowanie i wnioski końcowe

11.1 Osiągnięte rezultaty i wkład inżynierski

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie, implementacja oraz walidacja kompletnego, rozproszonego systemu umożliwiającego zdalne i wielodostępne zarządzanie procesami druku 3D z wykorzystaniem zaawansowanych technologii chmurowych, paradygmatu mikroserwisów oraz platformy sprzętowej Raspberry Pi pełniącej rolę lokalnego agenta orkiestracji. Założony cel badawczo-projektowy został w pełni osiągnięty poprzez wytworzenie, wdrożenie i przetestowanie ekosystemu *AddiPi*, który w sposób bezszwowy integruje warstwę sprzętową, oprogramowanie brzegowe, aplikację webową oraz rozproszoną infrastrukturę chmurową.

W ramach realizacji projektu inżynierskiego opracowano i zaimplementowano następujące komponenty oraz mechanizmy:

- **Rozproszona architektura mikroserwisowa** — struktura backendowa złożona z sześciu autonomicznych serwisów komunikujących się hybrydowo za pośrednictwem synchronicznych interfejsów REST API oraz asynchronicznej magistrali komunikatów *Azure Service Bus*,
- **Scentralizowany podsystem bezpieczeństwa** — model uwierzytelniania i autoryzacji bazujący na standardzie podwójnych tokenów JWT, integrujący mechanizm bezpiecznego odświeżania sesji, dwuetapową weryfikację adresów e-mail w domenie akademickiej oraz kontrolę dostępu opartą na rolach (RBAC),
- **Mechanizm orkiestracji i harmonogramowania zadań** — zaawansowany algorytm zarządzania kolejką produkcyjną, pozwalający zarówno na natychmiastową alokację zasobów sprzętowych, jak i długoterminowe planowanie kalendarzowe procesów przyrostowych,
- **Integracja brzegowa IoT** — powiązanie platformy sprzętowej z usługą *Azure IoT Hub*, umożliwiające synchroniczne sterowanie drukarką 3D za pomocą wywołań metod bezpośrednich (ang. *Direct Methods*) w standardzie RPC,
- **System monitorowania w czasie rzeczywistym** — mechanizm akwizycji danych

telemetrycznych urządzenia (metryki termiczne i statusy mechaniczne) oraz asynchronicznej dystrybucji strumienia wideo z kamery CSI w formacie MJPEG,

- **Wielojęzyczna aplikacja kliencka** — responsywna warstwa prezentacji zbudowana w oparciu o bibliotekę *React* oraz język TypeScript, wspierająca architekturę dynamicznych motywów graficznych (jasny i ciemny) oraz wzorzec projektowy RWD,
- **Infrastruktura jako kod (IaC)** — deklaratywne szablony konfiguracyjne narzędzia *Terraform*, automatyzujące procesy aprowizacji i replikacji zasobów chmurowych w architekturze hybrydowej *Microsoft Azure* oraz *Oracle Cloud Infrastructure*,
- **Zautomatyzowany potok wdrożeniowy (CI/CD)** — konfiguracja środowisk potokowych *GitHub Actions* zintegrowanych z rejestrem GHCR oraz automatycznym demonem aktualizacyjnym *Watchtower* w produkcyjnym środowisku kontenerowym,
- **Projekt mechaniczny CAD** — zoptymalizowana pod kątem tolerancji wymiarowych obudowa mikrokomputera wraz z przegubowym ramieniem pozycjonującym kamerę, zaprojektowana w programie *SolidWorks* i przygotowana do wytwarzania przyrostowego (FDM),
- **Automatyzacja i konfiguracja węzła brzegowego** — autorskie skrypty powłoki Bash zintegrowane z demonem *systemd* oraz systemowym harmonogramem *cron*, zabezpieczające bezobsługowy rozruch sieciowy i stabilność tunelu *Cloudflare Tunnel*.

Ostatecznie w ramach pracy powstał kompletny, w pełni funkcjonalny prototyp rozproszonego systemu zdalnego zarządzania autonomicznym stanowiskiem druku trójwymiarowego. Gotowe rozwiązanie zostało pomyślnie wdrożone i poddane długoterminowej walidacji w rzeczywistym środowisku pracy z fizyczną drukarką 3D. Przeprowadzone testy integracyjne oraz użytkowe potwierdziły wysoką niezawodność, bezpieczeństwo oraz bezobsługowość całego ekosystemu, udowadniając poprawność przyjętych założeń inżynierskich oraz wysoką efektywność wybranych technologii chmurowych i brzegowych.

11.2 Analiza napotkanych trudności projektowo-implementacyjnych

Realizacja rozproszonego systemu *AddiPi* wiązała się z koniecznością identyfikacji i rozwiązania szeregu zaawansowanych problemów technicznych. Wynikały one zarówno z heterogenicznego charakteru architektury wielochmurowej, jak i z fizycznej integracji zewnętrznych usług chmurowych z urządzeniami brzegowymi Internetu Rzeczy (IoT).

- **Integracja rozproszonego i heterogenicznego środowiska chmurowego**

Jednym z największych wyzwań architektonicznych było zapewnienie stabilnej orkiestracji komponentów działających w odmiennych środowiskach wykonawczych. Ekosystem wykorzystuje jednocześnie zasoby *Microsoft Azure* (*Cosmos DB*, *Blob Storage*, *Service Bus*, *IoT Hub*), instancję maszyn wirtualnych *Oracle Cloud Infrastructure*, platformę *Vercel* oraz fizyczny mikrokomputer brzegowy działający wewnątrz sieci lokalnej. Konieczne było zaprojektowanie spójnego modelu komunikacji, który uwzględniałby fluktuacje sieciowe, opóźnienia pakietów oraz skrajnie odmienne standardy uwierzytelniania dostawców. Wyzwanie to zminimalizowano poprzez wdrożenie hybrydowej architektury opartej na synchronicznych interfejsach REST API oraz asynchronicznej wymianie komunikatów z wykorzystaniem kolejki brokera, uzupełnionej o autorskie mechanizmy ponawiania żądań.

- **Spójność i normalizacja reprezentacji czasu**

Poszczególne warstwy systemu natywnie operowały na zróżnicowanych standardach zapisu dat i czasu. Informacje generowane przez silnik bazy danych, usługi backendowe oraz biblioteki frontendowe nie zawsze cechowały się identyczną strukturą, co generowało błędy w interpretacji i renderowaniu harmonogramów zadań. Problem ten rozwiązano na poziomie warstwy prezentacji poprzez implementację scentralizowanego modułu normalizacji danych chronologicznych, wykorzystującego autorską funkcję `normalizeDateString`.

- **Koordinacja sekwencyjności startu mikroservisów**

W fazie wdrażania środowiska kontenerowego zidentyfikowano, że natywna dyrektywa `depends_on` w konfiguracji *Docker Compose* gwarantuje wyłącznie kolejność inicjalizacji procesów, nie weryfikując rzeczywistej gotowości operacyjnej API do obsługi żądań sieciowych. Prowadziło to do błędów połączeń w sytuacji, gdy serwisy zależne próbowały komunikować się z komponentami będącymi jeszcze w

fazie wewnętrznego rozruchu. Wyzwanie to zaadresowano poprzez zaimplementowanie mechanizmów diagnostycznych stanu działania (ang. *health checks*) oraz procedurę ponawiania prób z wykładniczym czasem oczekiwania (ang. *retry with exponential backoff*) po stronie klientów HTTP.

- **Kompatybilność systemów modułów CommonJS oraz ESM**

Podczas integracji backendu z bibliotekami SDK *Azure IoT* w środowisku Node.js napotkano na konflikty struktur modułów *CommonJS* oraz *ECMAScript Modules* (ESM). Wybrane pakiety zależne eksportowały swoje składowe w sposób silnie uzależniony od kontekstu uruchomieniowego. W celu zapewnienia pełnej kompatybilności i stabilności kodu, w strukturze usługi *Printer Service* zaimplementowano mechanizm defensywnej detekcji i dynamicznej ekstrakcji dostępnych interfejsów eksportu.

- **Udostępnienie strumienia wideo poza barierę sieci lokalnej**

Ponieważ serwer *OctoPrint* operuje wewnątrz lokalnej sieci prywatnej, strumień wizyjny MJPEG nie był domyślnie trasowany do publicznego Internetu, co uniemożliwiało jego odbiór przez aplikację webową. Dodatkową barierę stanowiła zmienność adresów URL przy każdej renegotjacji połączenia tunelowego. Wyzwanie to rozwiązano za pomocą technologii *Cloudflare Tunnel* połączonej z autonomiczną usługą *Video Service*, która działa jako dynamiczny rejestrator aktualnego punktu końcowego wideo.

- **Obsługa opóźnień w synchronicznej komunikacji IoT**

Sterowanie parametrami fizycznej maszyny przy użyciu metod bezpośrednich (ang. *Direct Methods*) generowało opóźnienia rzędu kilku sekund w warunkach gorszej przepustowości sieciowej węzła brzegowego. Zjawisko to zostało uwzględnione na poziomie projektowania interfejsu użytkownika (UX) poprzez wprowadzenie asynchronicznych stanów ładowania, blokad przycisków przed wielokrotnym kliknięciem (ang. *debouncers*) oraz deterministycznej obsługi przekroczenia czasu oczekiwania (ang. *timeouts*).

Pomimo opisanych trudności, wszystkie zidentyfikowane bariery technologiczne zostały pomyślnie wyeliminowane na etapie implementacji. Pozwoliło to na uzyskanie w

pełni stabilnego, rozproszonego systemu, który w sposób bezwzględny spełnia wszystkie założenia inżynierskie określone w wymaganiach projektu.

11.3 Ograniczenia przyjętego rozwiązania

Pomimo pełnego spełnienia założonych wymagań projektowych, opracowany system posiada kilka ograniczeń technicznych wynikających z przyjętych kompromisów architektonicznych oraz specyfiki wybranych technologii.

- **Skala stanowiska produkcyjnego** — system został zoptymalizowany pod kątem obsługi pojedynczego urządzenia. Choć architektura pozwala na skalowanie, obecna implementacja dedykowana jest dla jednej kolejki zadań i jednego aktywnego węzła brzegowego.
- **Zależność od infrastruktury zewnętrznej** — podgląd wideo opiera się na bezpłatnym wariancie `trycloudflare.com`, co uniemożliwia przypisanie statycznego adresu URL i może generować okresowe przerwy w dostępności usługi sieciowej.
- **Limity darmowych planów chmurowych** — wdrożenie produkcyjne bazuje na bezpłatnych pakietach zasobów (*Oracle Always Free* oraz *Azure Free Tier*). Posiadają one sztywne limity wydajnościowe i pojemnościowe, wymagające migracji do planów komercyjnych przy wyższym obciążeniu.
- **Punkt pojedynczej awarii (SPOF)** — komunikacja z urządzeniem zależy bezpośrednio od stabilności serwera *OctoPrint* oraz mikrokomputera. Awaria warstwy brzegowej całkowicie paraliżuje proces realizacji nowych zleceń.
- **Brak statycznej walidacji kodu sterującego** — system nie realizuje automatycznej weryfikacji poprawności struktury plików G-code przed alokacją w kolejce. Walidacja poprawności technologicznej pliku spoczywa wyłącznie na operatorze.
- **Ograniczone pokrycie testami** — bieżąca wersja prototypowa nie została wyposażona w kompleksowy, zautomatyzowany zestaw testów integracyjnych typu *end-to-end*, który pokrywałby pełen cykl życia zadania.

11.4 Perspektywy i potencjalne kierunki dalszego rozwoju systemu

Opracowany system *AddiPi* został zaprojektowany z zachowaniem paradygmatu modularności oraz separacji odpowiedzialności, co zapewnia wysoką elastyczność i podatność na dalszą rozbudowę — zarówno w obszarze funkcjonalnym, jak i architektonicznym. Jako kluczowe, perspektywiczne kierunki rozwoju platformy zidentyfikowano następujące zagadnienia:

- **Obsługa środowiska wielourzędzeniowego (farma drukarek)** — rozszerzenie obecnej struktury dokumentów JSON o unikalne identyfikatory fizycznych maszyn oraz powiązanie z nimi odrębnych, izolowanych potoków kolejkowych, co umożliwi scentralizowane zarządzanie rozproszonym parkiem maszynowym z poziomu pojedynczej instancji aplikacji,
- **Autonomiczny system powiadomień natychmiastowych** — integracja architektury sieciowej z technologiami *Web Push* lub infrastrukturą *Firebase Cloud Messaging* (FCM) w celu natywnego informowania operatorów o zmianach stanów zadania (rozpoczęcie, sukces, awaria) bez konieczności utrzymywania aktywnej sesji w przeglądarce,
- **Statyczna analiza plików G-code przed wydrukiem** — implementacja modułu analizy składniowej kodu sterującego na etapie jego przesyłania do *Files Service*, co pozwoliłoby na precyzyjne, niezależne szacowanie czasu druku, wolumetrycznego zużycia filamentu oraz automatyczne wykrywanie błędów konfiguracji slicera,
- **Detekcja anomalii produkcyjnych z wykorzystaniem sztucznej inteligencji** — integracja strumienia wideo MJPEG z modelami uczenia maszynowego (np. konwolucyjnymi sieciami neuronowymi) w celu automatycznego wykrywania krytycznych wad druku w czasie rzeczywistym, takich jak odklejenie warstwy bazowej od stołu, zjawisko utraty spójności (ang. *spaghetti failure*) czy zablokowanie ekstrudera,
- **Rozbudowa mechanizmów federacyjnego uwierzytelniania** — implementacja protokołów *OAuth 2.0* oraz *OpenID Connect* (OIDC), co umożliwi bezpieczną integrację z zewnętrznymi dostawcami tożsamości (ang. *Identity Providers*), takimi jak konta *Microsoft*, *Google* lub *GitHub*,

- **Dedykowana aplikacja mobilna** — implementacja natywnej lub hybrydowej warstwy prezentacji (np. w technologii *React Native*), co podniosłoby ergonomię monitorowania telemetrii urządzenia na ekranach przenośnych,
- **Skalowanie i orkiestracja infrastruktury produkcyjnej** — przeniesienie struktury kontenerowej do środowiska orkiestracji *Kubernetes* (K8s) w celu wdrożenia mechanizmów automatycznego skalowania pionowego i poziomego, zaawansowanego równoważenia obciążenia (ang. *load balancing*) oraz scentralizowanego routingu usług,
- **Rozszerzenie warstwy jakości i odporności oprogramowania** — uzupełnienie kodu źródłowego o pełne pokrycie testami automatycznymi, generowanie żywej dokumentacji API w standardzie *OpenAPI/Swagger* oraz wdrożenie mechanizmów ochrony przed przeciążeniem interfejsów, takich jak *rate limiting* czy warstwa buforowania *Redis*.

11.5 Wnioski końcowe

Przeprowadzone prace projektowe, implementacyjne oraz walidacyjne potwierdziły, że integracja rozproszonej architektury mikroservisowej z usługami chmurowymi w modelu PaaS (*Platform as a Service*) stanowi wysoce efektywny paradygmat budowy systemów zarządzania autonomicznymi stanowiskami druku 3D. Wszystkie założone we wstępie cele pracy zostały w pełni zrealizowane. Opracowany system *AddiPi* umożliwia w pełni bezpieczne, zdalne przesyłanie plików z kodem sterującym, elastyczne harmonogramowanie zadań, ciągłe monitorowanie parametrów środowiskowych procesu oraz orkiestrację urządzenia z poziomu uniwersalnego interfejsu webowego.

Zastosowanie architektury zorientowanej na mikroservisy okazało się właściwym i uzasadnionym wyborem projektowym dla systemu tej klasy. Dekompozycja funkcjonalna na niezależne komponenty uprościła rozwój poszczególnych warstw logicznych, zwiększyła ogólną modularność rozwiązania oraz umożliwiła bezprzerwowe, asynchroniczne wdrażanie aktualizacji (ang. *zero-downtime deployment*). Jednocześnie realizacja projektu wykazała, że podejście mikroservisowe generuje dodatkowy narzut architektoniczny (ang. *overhead*), związany z koniecznością zapewnienia spójności danych, scentralizowanego zarządzania konfiguracją środowiskową oraz monitorowania rozpro-

szonych potoków komunikacyjnych.

Wykorzystanie ekosystemu *Microsoft Azure* pozwoliło na sprawną implementację kluczowych mechanizmów platformy bez konieczności ponoszenia kosztów budowy i utrzymania własnej infrastruktury serwerowej (ang. *on-premise backend*). Krytyczne znaczenie dla stabilności systemu miało wdrożenie usług *Azure IoT Hub*, *Azure Service Bus* oraz *Azure Cosmos DB*. Komponenty te umożliwiły bezbłędną realizację asynchronicznego routingu komunikatów, synchronicznego sterowania urządzeniem brzegowym przez protokół RPC oraz bezpiecznej persystencji obiektów JSON.

Realizacja warstwy sprzętowej potwierdziła, że mikrokomputer Raspberry Pi zintegrowany z serwerem *OctoPrint* stanowi stabilną, skalowalną i elastyczną platformę pośredniczącą pomiędzy fizyczną kinematyką drukarki 3D a systemami rozproszonymi. Opracowany komponent *AddiPi Agent* zabezpiecza poprawną i dwukierunkową wymianę informacji, gwarantując zarówno niskopoziomowe sterowanie maszyną, jak i ciągłą akwizycję danych telemetrycznych w czasie rzeczywistym.

Uzyskane rezultaty aplikacyjne wskazują, że system *AddiPi* posiada wysoki potencjał wdrożeniowy i może znaleźć realne zastosowanie nie tylko w strukturach akademickich laboratoriów studenckich, lecz również w placówkach badawczo-rozwojowych, szkołach technicznych, biurach projektowych oraz przedsiębiorstwach wykorzystujących współdzielone zasoby wytwarzania przyrostowego. Modułowa topologia rozwiązania pozostawia pełną swobodę w zakresie dalszej ewolucji systemu, umożliwiając implementację nowych funkcji oraz orkiestrację większej liczby maszyn bez konieczności modyfikacji lub przebudowy istniejących komponentów rdzenia.

Literatura

- [1] M. Jones, J. Bradley, and N. Sakimura. RFC 7519: JSON Web Token (JWT). Technical report, IETF, 2015.
- [2] Gina Häußge. Octoprint – The snappy web interface for your 3D printer. <https://octoprint.org>. Dostęp: 2026.
- [3] Microsoft Corporation. Typescript – JavaScript With Syntax For Types. <https://www.typescriptlang.org>. Dostęp: 2026.
- [4] OpenJS Foundation. Node.js – JavaScript runtime built on Chrome’s V8 engine. <https://nodejs.org>. Dostęp: 2026.
- [5] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>. Dostęp: 2026.
- [6] OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>. Dostęp: 2026.
- [7] Microsoft Corporation. Azure Cosmos DB documentation. <https://learn.microsoft.com/en-us/azure/cosmos-db/>. Dostęp: 2026.
- [8] Microsoft Corporation. Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>. Dostęp: 2026.
- [9] Microsoft Corporation. Azure Service Bus documentation. <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>. Dostęp: 2026.
- [10] Microsoft Corporation. Azure IoT Hub documentation. <https://learn.microsoft.com/en-us/azure/iot-hub/>. Dostęp: 2026.
- [11] Meta Platforms, Inc. React – The library for web and native user interfaces. <https://react.dev>. Dostęp: 2026.
- [12] Evan You et al. Vite – Next Generation Frontend Tooling. <https://vitejs.dev>. Dostęp: 2026.
- [13] Poimandres. Zustand – Bear necessities for state management in React. <https://github.com/pmndrs/zustand>. Dostęp: 2026.

- [14] Tailwind Labs Inc. Tailwind CSS – Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com>. Dostęp: 2026.
- [15] Docker Inc. Docker Documentation. <https://docs.docker.com>. Dostęp: 2026.
- [16] HashiCorp. Terraform — infrastructure as code. <https://www.terraform.io>. Dostęp: 2026.
- [17] Oracle Corporation. Oracle cloud infrastructure — free tier. <https://www.oracle.com/cloud/free/>. Dostęp: 2026.
- [18] Dassault Systèmes. SOLIDWORKS – 3D CAD Design Software. <https://www.solidworks.com>. Dostęp: 2026.
- [19] 219 Design. Case for Raspberry Pi 4 — GrabCAD Library. <https://grabcad.com/library/case-for-raspberry-pi-4-1>. Licencja CC BY 4.0, Dostęp: 2026.
- [20] Cloudflare Inc. Cloudflare Tunnel — Securely connect your infrastructure. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>. Dostęp: 2026.
- [21] containrrr. Watchtower — automatically update docker containers. <https://containrrr.dev/watchtower/>. Dostęp: 2026.
- [22] Vercel Inc. Vercel — build and deploy the best web experiences with the frontend cloud. <https://vercel.com>. Dostęp: 2026.